

**UNIVERSITY OF TRANSPORT AND COMMUNICATIONS
FACULTY OF INTERNATIONAL EDUCATION**



**GRADUATION PROJECT
DEGREE: ENGINEER**

Topic
**Research and Application of Diffusion-Based
Voice Conversion for Vietnamese Speech**
by

DO VIET HOANG

HIGH-QUALITY PROGRAM OF ENGLISH – VIETNAMESE
INFORMATION TECHNOLOGY
2021 - 2026 (Course 62)

HANOI - JUNE 2026

**UNIVERSITY OF TRANSPORT AND COMMUNICATIONS
FACULTY OF INTERNATIONAL EDUCATION**



GRADUATION PROJECT

Submitted in partial fulfillment of the requirements for the engineer's degree

HIGH-QUALITY PROGRAM OF ENGLISH – VIETNAMESE
INFORMATION TECHNOLOGY
2021 - 2026 (Course 62)

by

DO VIET HOANG

Under supervision of Dr. Lai Manh Dung

HA NOI - JUNE 2026

ASSURANCE STATEMENT

I hereby assure that this graduation project is my own original work and has been completed under the supervision of Dr. Lai Manh Dung. All information, data, and materials obtained from other sources have been fully acknowledged and properly referenced.

I confirm that this project has not been submitted, in whole or in part, for any other degree or qualification. I accept full responsibility for the accuracy and authenticity of the contents presented in this project.

Hanoi, 1st June, 2026

Do Viet Hoang

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to the Faculty of International Education and the Faculty of Information Technology, University of Transport and Communications for providing me with the fundamental knowledge and the opportunity to conduct this research.

I am especially grateful to my supervisor, Dr. Lai Manh Dung, whose insightful guidance and development orientations have been invaluable throughout the development of this project.

I also want to thank my family and friends for their support and belief in me. Their encouragement has been a great source of motivation during the most challenging stages of this journey.

Hanoi, 1st June, 2026

Do Viet Hoang

TABLE OF CONTENTS

ASSURANCE STATEMENT	i
ACKNOWLEDGEMENTS	ii
TABLE OF CONTENTS.....	iii
LIST OF SYMBOLS AND ABBREVIATIONS.....	vi
LIST OF TABLES	vii
LIST OF FIGURES	viii
INTRODUCTION	1
CHAPTER 1: OVERVIEW AND ANALYSIS	4
1.1. Overview of The Project.....	4
1.2. Voice Conversion.....	5
1.2.1. Definition of Voice Conversion.....	5
1.2.2. Types of Voice Conversion.....	5
1.3. Singing voice conversion	7
1.4. Speech Representations	8
1.4.1. Mel-spectrogram.....	8
1.4.2. Fundamental Frequency (F_0)	10
1.4.3. Linguistic Content (Units).....	10
1.4.4. Volume	12
1.4.5. Speaker identity	13
1.5. Generative Models and Diffusion-SVC Architecture.....	13
1.5.1. Denoising Diffusion Probabilistic Models (DDPM).....	13
1.5.2. Speakers Disentanglement with ContentVec.....	14
1.5.3. Shallow Diffusion Mechanism	14
1.6. Vietnamese Language in Speech and Voice Processing.....	14
1.6.1. Physical Foundations of Acoustic Signal	14
1.6.2. Characteristics of Vietnamese Speech.....	15
1.6.3. Vietnamese Speech Corpora.....	15
1.6.4. Challenges and Recent Advancements in Vietnamese Voice Conversion...	16
1.7. Chapter summary.....	17

CHAPTER 2: SYSTEM ARCHITECTURE AND METHODOLOGY	18
2.1. System Overview.....	18
2.2. Feature Extraction and Disentanglement	19
2.2.1. Linguistic Content Extraction	19
2.2.2. Fundamental Frequency and Volume Extraction	19
2.2.3. Speaker Representation (Speaker Embedding)	20
2.3. Feature Encoders and Acoustic(Naive) Model.....	20
2.3.1. Contentvec	20
2.3.2. Auxiliary Acoustic Model (Naive Predictor).....	22
2.4. Generative Mechanism.....	23
2.4.1. Denoising Diffusion Probabilistic Models	23
2.4.2. Shallow Diffusion Solution	24
2.5. Waveform Generation: NSF-HiFiGAN Vocoder.....	25
2.6. Training and Inference Pipelines	26
2.6.1. Training Pipeline	27
2.6.2. The Inference Pipeline (Voice Conversion)	30
2.7. Baseline models.....	32
2.8. Chapter Summary	33
CHAPTER 3: EXPERIMENTS AND EVALUATION.....	34
3.1. Experimental Setup and Datasets	34
3.1.1. Hardware and Software Environment	34
3.1.2. Dataset Description	35
3.2. Data Preprocessing	37
3.2.1. Contentvec	37
3.2.2. Diffusion.....	41
3.2.3. NSF-HiFiGan	44
3.3. Fine-tuning Process and Evaluation	44
3.3.1. Evaluation methods	44
3.3.2. Fine-tuning pipeline.....	47
3.3.3. Final results	60

3.4. Other personal modifications and clarifications.....	63
3.4.1. Contentvec	63
3.4.2. Diffusion.....	63
3.4.3. Evaluation methods	65
3.5. Chapter summary.....	65
CONCLUSIONS AND RECOMMENDATIONS	66
REFERENCES	68

LIST OF SYMBOLS AND ABBREVIATIONS

Abbreviations	Interpretation
ASR	Automatic Speech Recognition
CNN	Convolutional Neural Network
DDPM	Denoising Diffusion Probabilistic Models
F0	Fundamental Frequency/Pitch
GAN	Generative Adversarial Network
MCD	Mel Cepstral Distance
MOS	Mean Opinion Score
MSE	Mean Squared Error
NISQA	Non-Intrusive Speech Quality Assessment
NSF	Neural Source Filter
PESQ	Perceptual Evaluation of Speech Quality
RMS	Root Mean Square
RMSE	Root Mean Square Error
SC	Spectral Convergence
STFT	Short-Time Fourier Transform
STOI	Short-Time Objective Intelligibility
SVC	Singing Voice Conversion
SVS	Singing Voice Synthesis
TTS	Text To Speech
VC	Voice Conversion
WER	Word Error Rate

LIST OF TABLES

Table 1.1: Classification of Voice Conversion based on Training Data Structure	6
Table 1.2: Classification of Voice Conversion based on Speaker Involvement	7
Table 1.3: Vietnamese syllable structure table	15
Table 3.1: Reference-based metrics of the finetuned SD (Compared to the original audios) (6 samples)	50
Table 3.2: Speaker similarity scores of the phase 1 pipeline (Averaged of 100 samples)	51
Table 3.3: Reference-based metrics between partial and full fine-tuning (Compared to the original audios, average of 6 samples)	55
Table 3.4: Reference-based metrics between checkpoints (Compared to the original audios, average of 6 samples)	58
Table 3.5: Reference-based metrics between fine-tuned and stock NSF-HiFiGan (Compared to the original audios, average of 6 samples)	59
Table 3.6: Reference-based metrics between final pipeline and stock pipeline(Phase 1). (Compared to the original audios, average of 10 samples)	62
Table 3.7: Speaker similarity scores of the final pipeline (Averaged of 100 samples) ..	62

LIST OF FIGURES

Figure 1.1: The typical flow of a voice conversion system [7]	5
Figure 1.2: An example of Narrowband Spectrogram (Formants).....	8
Figure 1.3: An example of Broadband Spectrogram (Harmonics).....	9
Figure 1.4: An example of F0 curve (Pitch)	10
Figure 1.5: An example of Formants	11
Figure 1.6: An example of Volume line (Intensity)	12
Figure 1.7: An example of reverse process [8].....	13
Figure 2.1: The overall structure of CONTENTVEC [11].....	20
Figure 2.2: A conceptual curve of how speaker information changes through the network layers.[11]	21
Figure 2.3: Pipeline overview of hn-sinc-NSF, HiFi-Gan and NSF-HiFiGan	26
Figure 2.4: Overview of training and inference pipelines [14]	27
Figure 2.5: Naive model's training pipeline	28
Figure 2.6: Diffusion model's training pipeline	29
Figure 2.7: Inference pipeline.....	31
Figure 3.1: Notes distribution of male subset.....	35
Figure 3.2: Length distribution of male subset.....	36
Figure 3.3: Notes distribution of female subset	36
Figure 3.4: Length distribution of female subset	37
Figure 3.5: First few entries of train.tsv (The same for valid.tsv).....	38
Figure 3.6: Cached file prepared for clustering.....	39
Figure 3.7: Kmean algorithm found the convergence at step 1143	39
Figure 3.8: First few entries of the dummyy dictionary	40
Figure 3.9: Contentvec dataset folder structure after the data is preprocessed	41
Figure 3.10: Diffusion dataset folder structure after the data is preprocessed	43
Figure 3.11: Baseline NISQA scores (Averaged)	48
Figure 3.12: Baseline WER scores (Percent, Averaged)	48
Figure 3.13: Training loss (left, 0.9 smoothing) and validation loss (right, 0.5 smoothing) of naive model. X axis is number of steps, Y axis is loss value.	49
Figure 3.14: Training loss (left, 0.5 smoothing) and validation loss (right, 0.5 smoothing) of diffusion model. X axis is number of steps, Y axis is loss value.	50
Figure 3.15: NISQA scores of the finetuned SD compared to the best baseline variant (6 samples)	50
Figure 3.16: Training loss (blue) and validation loss (red) of 500 and 100 cluster model (0.8 smoothing). X axis is number of steps, Y axis is validation loss.	52
Figure 3.17: NISQA scores between 100 and 500 clusters labeling	53
Figure 3.18: Additional parameter and logic for freezing the transformer layers	54

Figure 3.19: Training loss (blue) and validation loss (red) of Partial Fine-Tuning model (0.8 smoothing). X axis is number of steps, Y axis is validation loss.	54
Figure 3.20: NISQA scores between partial and full fine-tuning.....	55
Figure 3.21: An example of spectral smearing, full model.	56
Figure 3.22: The same vowel from partial model.	56
Figure 3.23: NISQA scores between checkpoints (2k-4k-6k steps).....	58
Figure 3.24: Total Validation loss of fine-tuned model (0.8 smoothing). X axis is number of steps, Y axis is validation loss.	59
Figure 3.25: NISQA scores between fine-tuned and stock NSF-HiFiGan.....	59
Figure 3.26: Training loss (left) and validation loss (right) of naive model. X axis is number of steps, Y axis is loss value, 0.8 smoothing.	60
Figure 3.27: Training loss (left) and validation loss (right) of diffusion model. X axis is number of steps, Y axis is loss value, 0.8 smoothing.	60
Figure 3.28: NISQA scores between final pipeline and baseline (averaged).....	61
Figure 3.29: WER scores between final pipeline and baseline (averaged).....	61
Figure 3.30: Proposed F0 extractor block	64

INTRODUCTION

In recent years, voice conversion (VC) have become important research topics in the fields of speech processing and artificial intelligence. These technologies aim to transform a source speaker’s voice into the voice of a target speaker while preserving the original linguistic content, timing, and prosodic characteristics. Voice conversion systems have a wide range of applications, including music production, virtual avatars, dubbing systems, speech synthesis, and assistive technologies.

Traditional approaches to voice conversion typically rely on statistical models, such as Gaussian Mixture Models, or neural network-based methods including sequence-to-sequence models and Generative Adversarial Networks (GANs). While these approaches can achieve acceptable results, they often suffer from limitations such as unstable training, loss of fine acoustic details, and oversmoothing in generated spectrograms.

More recently, diffusion models have emerged as a powerful generative modeling framework capable of producing high-quality outputs by gradually transforming noise into structured data through a denoising process. Diffusion-based models have demonstrated impressive performance in various generative tasks, including image generation, speech synthesis, and audio generation. In the context of voice conversion, diffusion models can significantly improve the quality and naturalness of generated audio by refining acoustic features through iterative denoising.

However, research on diffusion-based voice conversion systems for the Vietnamese language remains very limited. Vietnamese is a tonal language in which pitch variations are essential for distinguishing lexical meaning. As a result, preserving pitch accuracy and tonal information during the conversion process is particularly challenging and crucial.

Furthermore, unlike languages such as English, Japanese, or Chinese, most Vietnamese speech research has traditionally focused on tasks such as automatic speech recognition (ASR) and text-to-speech synthesis (TTS). For example, the VIVOS corpus has been widely used as a benchmark dataset for Vietnamese speech recognition research [1], and several open speech datasets have been developed to support Vietnamese text-to-speech systems [2][3][4][5]. These resources have contributed significantly to the development of Vietnamese speech technologies.

However, research on more advanced speech generation tasks, particularly speech-to-speech voice conversion and singing voice conversion, remains relatively limited for the Vietnamese language. Only recently have dedicated research efforts begun to appear

in this area. For instance, the VLSP 2025 workshop introduced the first shared task on Vietnamese voice conversion, providing a benchmark dataset and evaluation framework for this problem [6]. The emergence of this shared task indicates that Vietnamese voice conversion research is still in an early stage of development.

As a result, publicly available open-source systems and pretrained models specifically designed for Vietnamese speech-to-speech voice conversion remain scarce. This creates a gap between the rapid development of modern voice conversion techniques and their practical application to Vietnamese speech processing. Addressing this gap is therefore an important motivation for this project.

The main objective of this project is to research and apply a diffusion-based voice conversion model for Vietnamese speech and to some extent, singing data. In particular, this project investigates the application of the Diffusion-SVC framework, which combines a lightweight naive acoustic model with a shallow diffusion model in order to generate high-quality converted speech.

The specific objectives of this project include:

- Studying the theoretical foundations of voice conversion and diffusion-based generative models.
- Analyzing the architecture and working principles of the Diffusion-SVC framework.
- Implementing a voice conversion pipeline based on the Diffusion-SVC open-source repository.
- Applying the model to Vietnamese audio data in order to evaluate its feasibility for tonal language speech processing.
- Evaluating the quality and performance of the generated audio results.

Through these objectives, this project aims to explore the potential of diffusion-based approaches for Vietnamese voice conversion and contribute to the development of Vietnamese speech processing technologies.

This project focuses on the research and practical implementation of a diffusion-based voice conversion system using the Diffusion-SVC framework. The system utilizes a shallow diffusion approach that combines a naive acoustic model with a diffusion model to balance audio quality and computational efficiency.

The scope of this project includes studying the system architecture, feature extraction process, naive model prediction, shallow diffusion refinement, and waveform

synthesis using a neural vocoder. The implementation is based on an existing open-source framework which is analyzed and adapted for experimental purposes.

However, several limitations should be noted. The experiments are conducted using a limited Vietnamese dataset due to constraints in available training data and computational resources. Training diffusion models typically requires substantial GPU resources and long training times, which restricts the scale of experiments performed in this project. In addition, the evaluation of the system mainly focuses on qualitative listening tests and basic analysis rather than extensive large-scale benchmarking.

The implementation of this project, including source code modifications and model configurations, as well as demonstrations are available at the following repository. This work is built upon and adapted from several open-source projects, with full details provided in the repository's documentation:

URL: github.com/Just-a-lurker/BCDA_VC

This report consists of three chapters:

- Chapter 1: Overview and Analysis: Establishing the theoretical foundation of Voice Conversion and provides a comprehensive analysis of the challenges involved in adapting these systems for the Vietnamese language. Exploring the current state of neural speech synthesis, the complexities of Vietnamese tonality and the impact of limited high-quality data on model performance.

- Chapter 2: System architecture and methodology: Detailing the technical framework of the proposed pipeline, centering on the Shallow Diffusion-SVC architecture. Describing the integration of the encoder for feature extraction and the vocoder for waveform synthesis.

- Chapter 3: Experiments and evaluations: Documenting the experimental setup, including the collection and pre-processing of the Vietnamese vocal dataset. Presenting a comparative analysis between different training configurations, utilizing objective metrics such as MCD, PESQ, and F0 RMSE, alongside neural-based assessments like NISQA. The results highlight the trade-offs between mathematical optimization and perceptual audio quality.

- Conclusion and recommendations: Summarizing the key findings of the research, confirming the effectiveness of the Diffusion-based pipeline for Vietnamese voice conversion. Acknowledging the constraints regarding data diversity and computational resources, while offering strategic recommendations for future work, such as native Vietnamese pre-training and advanced denoising techniques.

CHAPTER 1: OVERVIEW AND ANALYSIS

Purpose of the chapter

This chapter presents the theoretical background related to speech processing and voice conversion systems. It also introduces key techniques used in modern voice generation models, including acoustic feature representations and diffusion-based generative models. These concepts provide the necessary foundation for understanding the framework used in this project. The reviewed background will support the analysis and system implementation described in the subsequent chapters.

1.1. Overview of The Project

Speech processing is a field of research that focuses on the analysis, synthesis, and transformation of human speech using computational methods and signal processing techniques. The goal of speech processing is to enable machines to understand, generate, and manipulate human speech signals in a way that is useful for various applications.

Modern speech processing systems typically operate by converting raw audio signals into structured feature representations that can be processed by machine learning models. Common acoustic representations include spectrograms, mel-spectrograms, pitch information, and learned speech embeddings extracted from pretrained models. These representations capture important characteristics of speech signals such as frequency content, temporal dynamics, and speaker-specific properties.

Several important research areas exist within speech processing, including ASR, TTS synthesis, speaker recognition, and voice conversion. Automatic speech recognition aims to convert spoken language into textual form, while text-to-speech synthesis generates speech from text input. Voice conversion focuses on modifying the speaker identity of speech while preserving the linguistic information contained in the original utterance [7].

Recent advances in deep learning have significantly improved the performance of speech processing systems. Neural network architectures are now widely used to model complex relationships between speech features and audio signals [7], enabling more natural and realistic speech generation and transformation.

1.2. Voice Conversion

1.2.1. Definition of Voice Conversion

Voice conversion refers to the process of transforming speech produced by a source speaker so that it sounds as if it were spoken by a target speaker while preserving the linguistic content of the original speech [7]. In other words, voice conversion modifies speaker-specific characteristics such as timbre, vocal tract properties, and speaking style while maintaining the phonetic and semantic information of the speech signal.

A typical voice conversion system attempts to separate speaker-independent information, such as linguistic content, from speaker-dependent information such as voice identity. After this separation, the system modifies the speaker-related features in order to generate speech that resembles the target speaker.

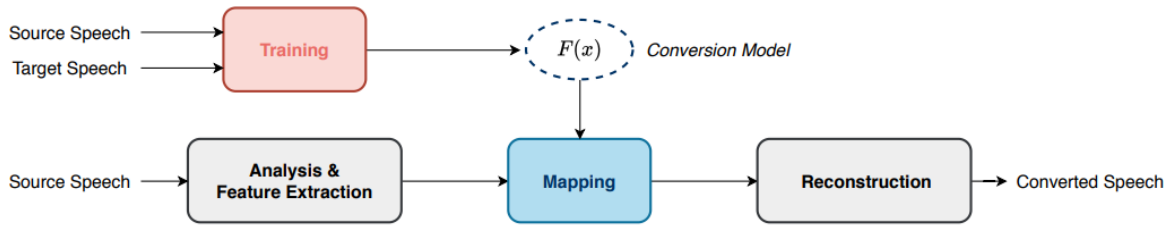


Figure 1.1: The typical flow of a voice conversion system [7]

Early voice conversion systems were primarily based on statistical techniques, including Gaussian Mixture Models and frequency warping methods. These approaches attempted to learn a mapping between acoustic features of the source speaker and those of the target speaker. However, such methods often produced speech with limited naturalness and suffered from oversmoothing effects in the generated acoustic features [7].

More recent approaches employ deep learning architectures, including encoder-decoder networks, variational autoencoders, GANs, and diffusion-based generative models. These models are capable of capturing more complex relationships between acoustic features and can generate higher quality and more natural-sounding converted speech.

1.2.2. Types of Voice Conversion

Voice conversion systems can be categorized according to different criteria, including the structure of the training data and the number of speakers involved in the conversion process.

One common classification distinguishes between parallel and non-parallel voice conversion systems:

Table 1.1: Classification of Voice Conversion based on Training Data Structure

Data Structure Type	Description	Advantages	Limitations
Parallel Voice Conversion	- Uses paired recordings where source and target speakers pronounce identical utterances. - Modern approaches utilize Sequence-to-Sequence architectures comprising an Encoder, Attention mechanism, and Decoder [7].	- Capable of learning a highly accurate direct mapping between the corresponding speech segments.	- Expensive, time-consuming, and highly difficult to collect, as it requires speakers to record identical sentences.
Non-parallel Voice Conversion	- Does not require aligned recordings. - Employs Auto-encoder architectures where the Encoder utilizes representation learning (often with Self-Attention) to decouple linguistic content, and the Decoder reconstructs speech using a target speaker embedding [7].	- Highly flexible and scalable, as the model can be trained on independently collected, unaligned speech data from various real-world sources.	- Requires highly complex feature disentanglement mechanisms to ensure accurate separation of content and timbre without parallel supervision.

Voice conversion systems can also be categorized based on the number of speakers involved:

Table 1.2: Classification of Voice Conversion based on Speaker Involvement

Conversion Type	Source Speaker(s)	Target Speaker(s)	Description
Many-to-One	Multiple	Single	The system is designed to convert speech from various different source speakers into one specific target voice.
One-to-Many	Single	Multiple	The system converts speech from a single source speaker into several different target voices.
Many-to-Many	Multiple	Multiple	A highly advanced unified model that supports bi-directional or multi-directional conversion between any source and target speakers within the dataset [7].

1.3. Singing voice conversion

Singing Voice Conversion is a specialized form of voice conversion that focuses on transforming a singing voice produced by a source singer so that it sounds as if it were performed by a target singer while preserving the original musical content, including lyrics, melody, and rhythm. Unlike traditional speech voice conversion, SVC must handle additional challenges related to musical structure, such as wide pitch variations, vibrato effects, and long sustained phonemes that frequently occur in singing.

SVC is closely related to other singing voice technologies such as singing voice synthesis (SVS). While SVS aims to generate singing voices from musical scores or text input, SVC directly transforms an existing singing recording into the voice of another singer [9]. Because SVC uses real singing audio as input, it must accurately preserve musical expression while modifying speaker-dependent characteristics such as timbre and vocal identity.

One of the key challenges in singing voice conversion is the large variation in pitch and dynamics present in singing signals. Compared to normal speech, singing voices often contain higher pitch ranges and stronger temporal variations, which makes feature modeling more complex. As a result, many SVC systems explicitly extract pitch-related features such as the fundamental frequency (F0) and use them as conditioning information during the conversion process.

Recent advances in deep learning have significantly improved the performance of SVC systems. Modern approaches often use neural networks to learn speaker-independent content representations and combine them with speaker embeddings and pitch features to generate target singing voices. These systems may employ architectures such as encoder-decoder networks, generative adversarial networks, or diffusion-based generative models [8].

Diffusion-based models have recently shown strong potential for audio generation tasks, including singing voice processing. For example, the DiffSinger system introduced a shallow diffusion mechanism to efficiently generate high-quality singing voice signals while reducing the number of diffusion steps required during inference [9]. This idea of using diffusion models to refine acoustic representations has influenced later research in diffusion-based voice conversion systems, including Diffusion-SVC, which applies similar principles to the task of singing voice conversion.

1.4. Speech Representations

In the context of voice conversion, selecting appropriate speech representations is crucial for achieving high-quality synthesis. Raw audio signals are high-dimensional and contain complex temporal dependencies, making them difficult to model directly. Therefore, most modern systems, including Diffusion-SVC, rely on intermediate acoustic features. In this section, Paraat is used to demonstrate the features.

1.4.1. Mel-spectrogram

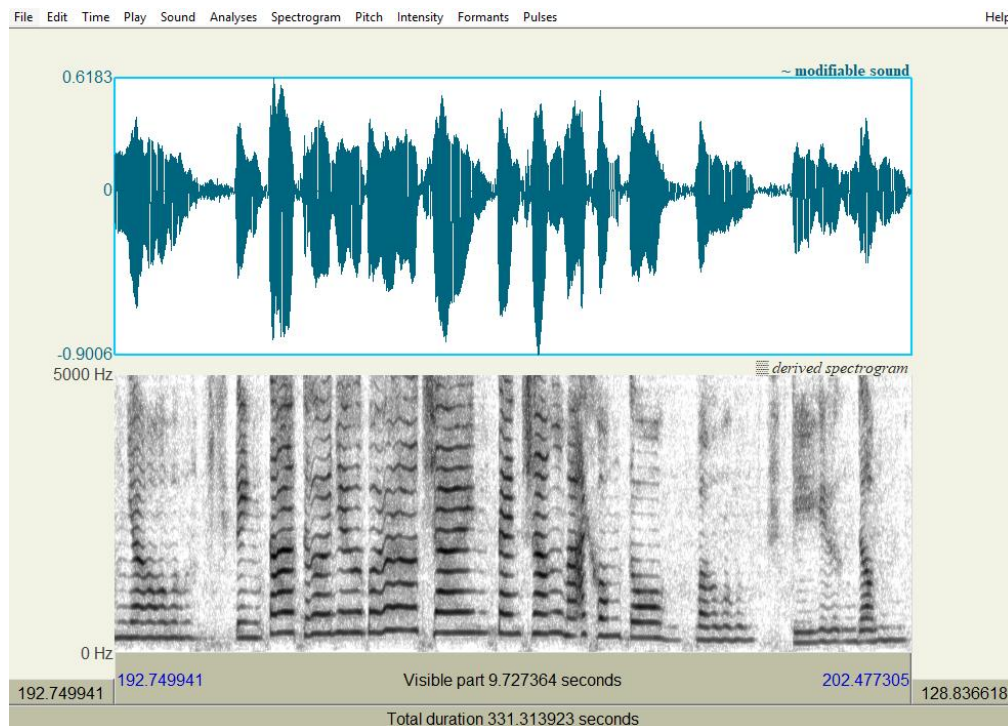


Figure 1.2: An example of Narrowband Spectrogram (Formants)

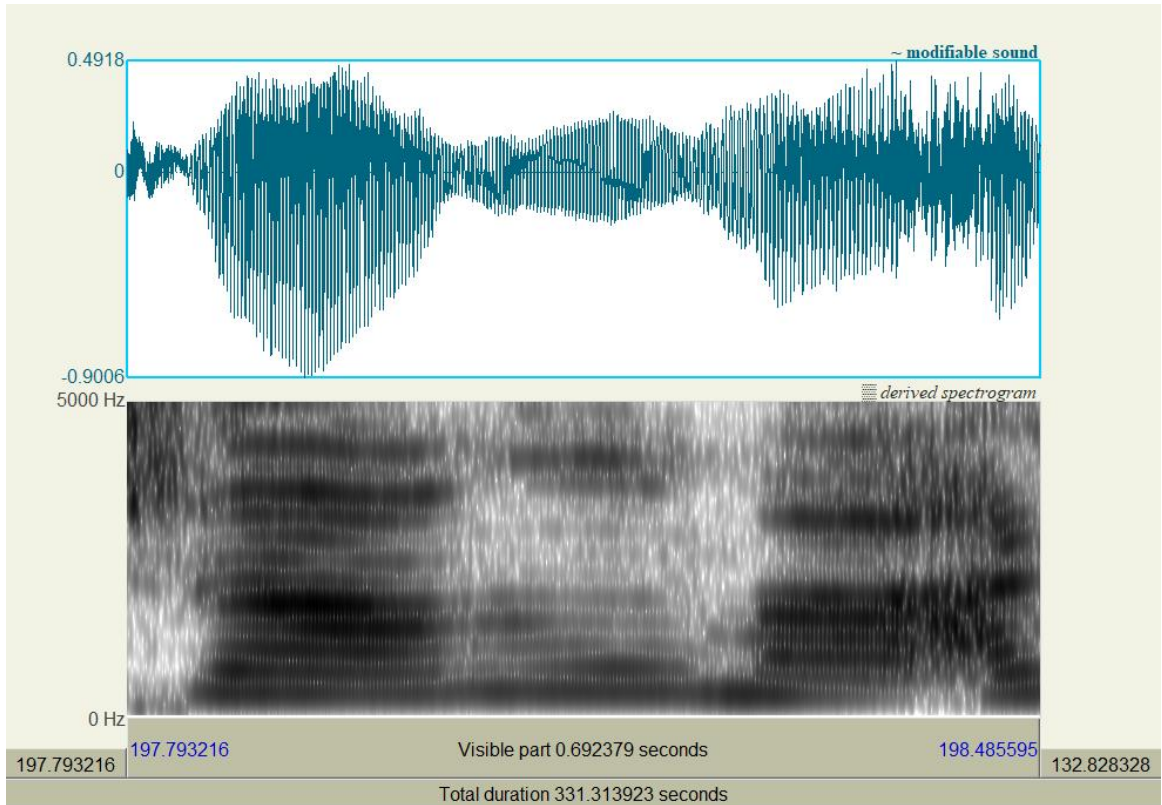


Figure 1.3: An example of Broadband Spectrogram (Harmonics)

The mel-spectrogram is one of the most widely used representations in generative speech modeling. It is generated by applying a Short-Time Fourier Transform (STFT) to the audio signal and mapping the resulting frequencies onto the Mel scale. This scale is designed to mimic the non-linear human perception of pitch, providing higher resolution for lower frequencies where human hearing is most sensitive.

Depending on the spectrogram window length, different acoustic characteristics can be emphasized. A narrowband spectrogram (0.025-0.04s) highlights pitch information, where the visible harmonic lines correspond to multiples of the fundamental frequency, enabling detailed analysis of vocal pitch and timbre. In contrast, a broadband spectrogram (0.005s) suppresses individual harmonics and instead emphasizes bold horizontal bands, known as formants, which correspond to frequency regions amplified by the vocal tract, including the oral cavity and throat.

In diffusion-based voice conversion, the mel-spectrogram often serves as the training target for the diffusion process [9]. It provides a compact representation of the spectral envelope, which carries information about the timbre and vocal tract characteristics of the speaker. By reconstructing the mel-spectrogram through a reverse diffusion process, then feed it to a decoder/vocoder like NSF-HiFiGan, the system can generate high-fidelity audio that sounds natural and clear.

1.4.2. Fundamental Frequency (F_0)

The fundamental frequency, denoted as F_0 , represents the physical property of the rate of vibration of the vocal folds and is perceived as the pitch of the voice. In the context of voice conversion, F_0 is a vital conditioning feature. Since voice identity is closely tied to the average pitch and prosodic patterns, the system must be able to adjust F_0 to match the target speaker's range while maintaining the original melody and intonation of the source speech.

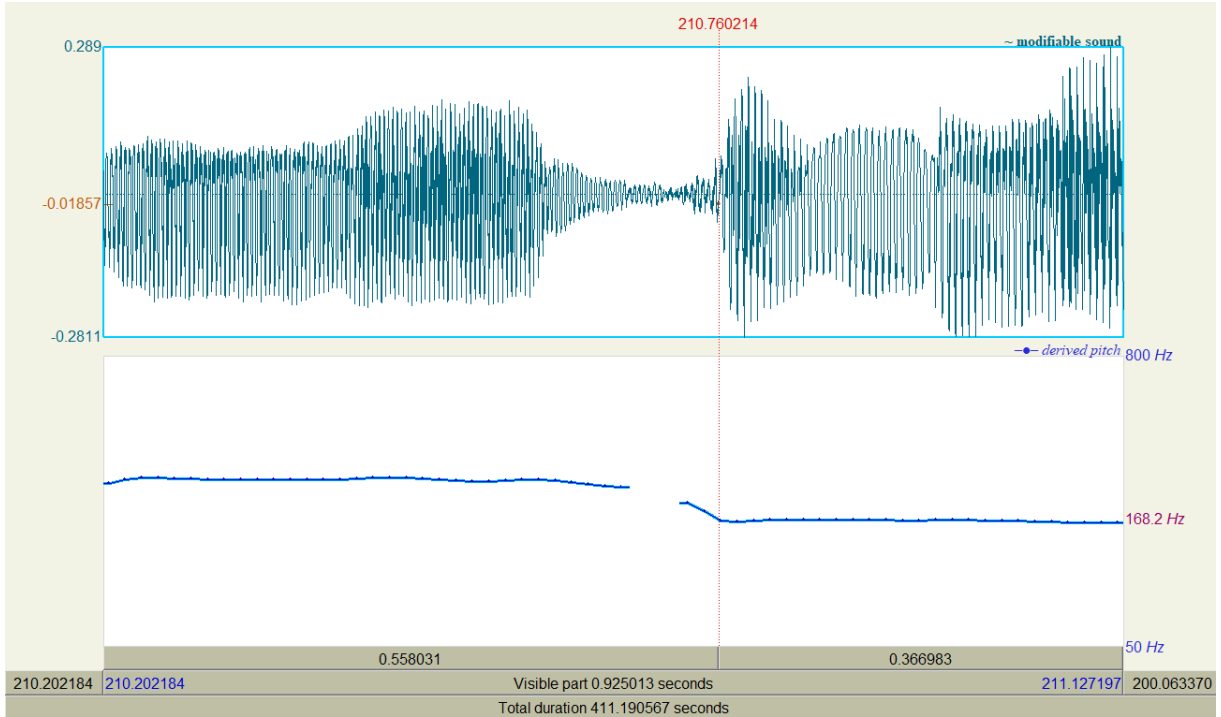


Figure 1.4: An example of F_0 curve (Pitch)

The higher the line, the higher the note. If this line is broken, those are unvoiced sections or fricatives like "s", "p", "t".

Many modern voice conversion frameworks, including Diffusion-SVC, extract F_0 explicitly using algorithms like Parallel WaveGAN or Harvest/Rmvpe/Dio. This extracted pitch information is then used as a guide for the model to ensure that the linguistic nuances and emotional expressions are preserved during the transformation of the speaker's identity.

1.4.3. Linguistic Content (Units)

To achieve high-quality voice conversion, the system must separate the linguistic content from the speaker's individual voice characteristics (style). Recent advances have introduced the use of self-supervised learning models, such as HuBERT or ContentVec, to extract this feature. The linguistic content can also be inspected as formants in softwares.

Figure 1.5: An example of Formants

The units are represented by rows of red dots run horizontally across the spectrogram. They are labeled F1, F2, F3, F4, etc....The spacing between these red dots determines what letter is heard. For example, in figure 1.4 from left to right: When the letter "i" is pronounced (The first rectangle), the F1 dots is lower and F2 dots are higher, and for the letter "u" (The second rectangle), it's the opposite. In the model, the units are essentially trying to simulate the structure of these red-dot patterns.

In the context of the VC framework, Units serve as the foundational backbone of the "Source-to-Target" mapping. These features are essentially high-dimensional vectors that capture the nuances of human articulation-such as the positioning of the tongue and the shape of the vocal tract-without being tethered to the physical pitch or the unique vocal texture of the source speaker. Depending on the model configuration, these units can be continuous, preserving a rich spectrum of phonetic detail, or quantized (discrete), which further enforces the "de-identification" process by mapping speech sounds into a fixed set of "codebook" entries. By stripping away the non-essential acoustic data, Units provide a clean, linguistic template that allows the generative model to focus entirely on applying the target speaker's unique "acoustic filter" over the provided linguistic structure.

1.4.4. Volume

Volume, frequently referred to as energy or loudness, represents the instantaneous amplitude of the speech signal over time. In voice conversion architectures, volume acts as a crucial dynamic conditioning feature that captures the rhythmic stress, emotional intensity, and breathing patterns of the source audio. While units dictate the linguistic content and F0 controls the melody, volume dictates the physical force and dynamic range behind the utterance.

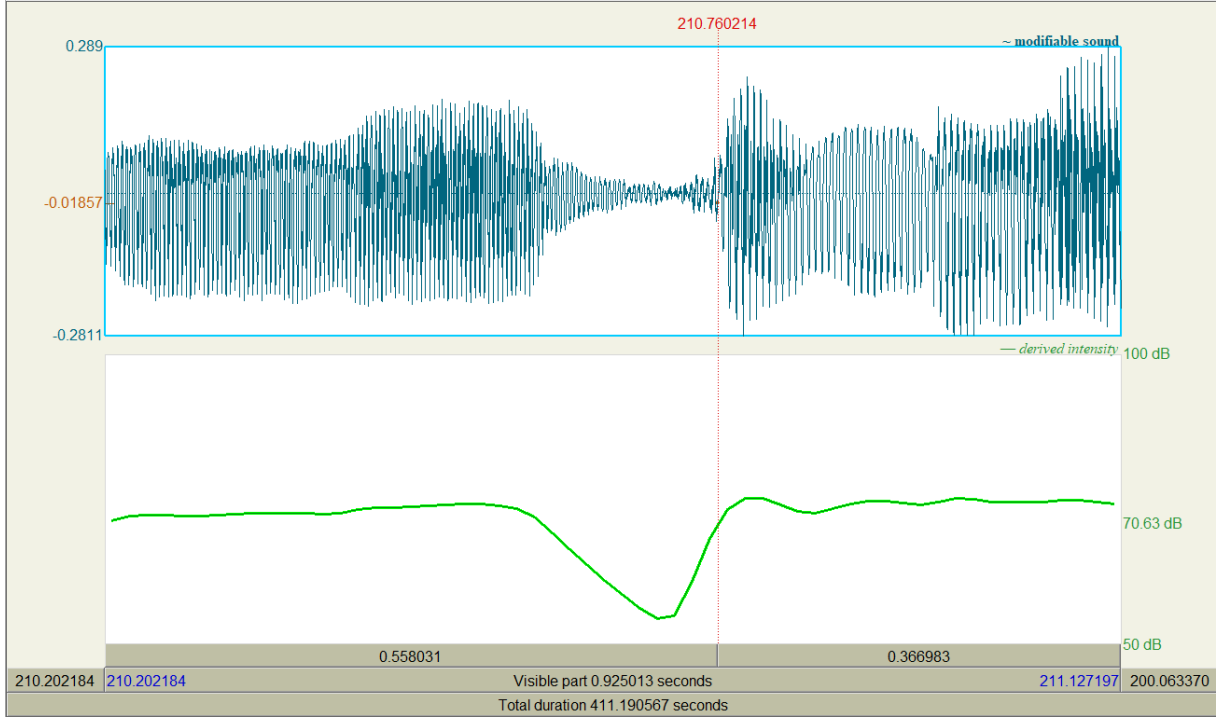


Figure 1.6: An example of Volume line (Intensity)

The line spikes high on vowels (like "a", "o") due to their high energy, and drops low on consonants or rests.

In advanced frameworks like Diffusion-SVC, volume is typically extracted as the Root Mean Square (RMS) energy or an amplitude envelope. Conditioning the generative model on this energy contour is essential for maintaining natural prosody. It allows the model to accurately reproduce subtle acoustic variations—such as whispers, emphatic syllables, or natural pauses—preventing the synthesized output from sounding robotic, flat, or generating artificial noise during silent intervals. By decoupling the energy dynamics from the speaker's timbre, the system ensures that the structural intensity of the source speech is faithfully preserved and seamlessly integrated with the target speaker's acoustic identity.

1.4.5. Speaker identity

Speaker identity is the unique vocal signature that makes a person’s voice sound like them. It is made of two parts: the physical shape of their throat and mouth (called timbre), and their personal speaking habits like accents or speed. One of the main goals of a voice conversion system is feature disentanglement, which means separating this identity from the actual words spoken and the pitch.

In modern architectures, this identity is typically extracted as a speaker embedding from the dataset. The system then uses this target embedding to condition the synthesis network, seamlessly convert the new speaker’s identity onto the source text and prosody.

1.5. Generative Models and Diffusion-SVC Architecture

Generative models have significantly advanced the field of voice conversion. While earlier approaches relied on architectures like GANs or variational autoencoders, diffusion models have emerged as a powerful alternative, offering superior training stability and audio fidelity. The Diffusion-SVC framework specifically leverages these models alongside advanced feature extraction to map acoustic features into high-quality mel-spectrograms.

1.5.1. Denoising Diffusion Probabilistic Models (DDPM)

Denoising Diffusion Probabilistic Models (DDPM) form the core generative engine of the system. A DDPM learns to synthesize data by reversing a gradual noise-addition process. The framework consists of a forward process and a reverse process.

The forward process is a Markov chain that progressively adds Gaussian noise to the original data distribution (such as a mel-spectrogram) over a series of steps, eventually transforming the signal into pure noise. The reverse process involves training a neural network to denoise the data step-by-step, starting from the noise and gradually recovering the target signal. By conditioning this reverse process on specific inputs - such as speech representations - the model can generate highly accurate and natural-sounding speech of the target speaker [8].

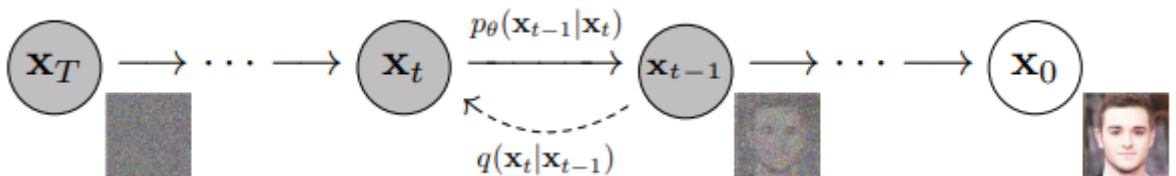


Figure 1.7: An example of reverse process [8]

1.5.2. Speakers Disentanglement with ContentVec

To ensure high-quality voice conversion, the diffusion model must be conditioned on linguistic content that are strictly decoupled from the source speaker's identity. Nowadays, VC systems utilize ContentVec, an advanced self-supervised speech representation model, for this purpose.

ContentVec improves upon traditional feature extractors by explicitly disentangling speaker information during its training phase. It achieves this through a teacher-student framework combined with voice conversion techniques that mask speaker identity, forcing the model to focus purely on linguistic content [11]. This robust separation prevents "speaker leakage," ensuring that the synthesized voice completely adopts the target speaker's timbre without retaining the source speaker's characteristics.

1.5.3. Shallow Diffusion Mechanism

While standard DDPMs generate excellent audio, their iterative reverse process requires hundreds or thousands of inference steps, making them computationally expensive and slow. To overcome this limitation, a shallow diffusion mechanism inspired by models like DiffSinger [9] is employed.

Instead of initiating the reverse process from pure Gaussian noise, VC systems utilize an auxiliary acoustic model (often a fast, lightweight network) to predict a coarse mel-spectrogram based on the input features (ContentVec embeddings and fundamental frequency). The forward diffusion process then adds only a limited number of noise steps, denoted as k , to this predicted spectrogram. Subsequently, the diffusion model performs the reverse denoising process for these k steps to refine the spectrogram and inject rich, natural acoustic details [9]. This shallow diffusion approach drastically reduces inference time while maintaining high audio quality, enabling the system to run efficiently.

1.6. Vietnamese Language in Speech and Voice Processing

Applying voice conversion technologies to the Vietnamese language introduces unique phonetic and linguistic challenges compared to English or other non-tonal languages. The development of robust Vietnamese voice conversion systems requires an understanding of these specific linguistic characteristics and relies heavily on the availability of high-quality, diverse speech corpora.

1.6.1. Physical Foundations of Acoustic Signal

The physical basis of sound in speech is characterized by three primary acoustic features:

- **Duration:** This relates to the length of the sound, depending on how long a phonetic unit is sustained over time.
- **Pitch (F0):** Pitch is determined by the vibration frequency of the vocal folds within a specific time unit, measured in Hertz (Hz). Higher pitches correspond to faster vibrations, while lower pitches correspond to slower vibrations. Pitch conveys critical information regarding speaker identity (gender, age), emotion, and, crucially in languages like Vietnamese, lexical meaning.
- **Energy (Intensity):** Energy relates to the loudness of the sound and depends on the amplitude of the sound wave. Larger amplitudes produce louder, more resonant sounds. Measured in decibels (dB), energy is a key factor in determining speech stress and emphasis.

1.6.2. Characteristics of Vietnamese Speech

Vietnamese is a highly complex language compared to many others due to its nature as a monosyllabic, tonal language. Every spoken syllable inherently carries a specific tone.

Table 1.3: Vietnamese syllable structure table

Tone			
Initial (Initial consonant / Onset)	Rhyme		
	Medial	Nucleus (main vowel)	Coda (final consonant)

Vietnamese language utilizes six distinct tones: level (ngang), acute (sắc), grave (huyền), smooth-rising (hỏi), creaky-rising (ngã), and heavy/drop (nặng). In Vietnamese, the fundamental frequency (F0) contour of a syllable directly determines its lexical meaning. A slight alteration in the pitch contour can completely change the semantic content of a word [10].

1.6.3. Vietnamese Speech Corpora

The progress of Vietnamese speech processing is closely tied to the development of public speech datasets. Early corpora were primarily designed for ASR, such as the VIVOS dataset, which provides 15 hours of recording from various speakers [1]. As the focus expanded to generative tasks, datasets like the FPT Open Speech Data were introduced to support TTS synthesis [2].

In recent years, the demand for multi-speaker and high-fidelity data for voice conversion has led to the creation of more comprehensive datasets. The infore1_25hours

dataset provides a substantial amount of high-quality Vietnamese speech [4], while the VieNeu-TTS dataset offers a multi-speaker corpus specifically tailored for modern generative speech models [3]. These datasets provide the essential acoustic diversity needed to train advanced models like Diffusion-SVC to generalize across different Vietnamese voices.

1.6.4. Challenges and Recent Advancements in Vietnamese Voice Conversion

For voice conversion systems, the tonal nature of the language presents a critical challenge. While converting a source speaker's voice to a target speaker's voice, the system must precisely modify the overall pitch range and timbre to match the target identity, but it must strictly preserve the relative F0 fluctuations that define the Vietnamese tones [10]. Failing to maintain these fine-grained pitch variations leads to semantic distortion and significantly reduces the intelligibility of the converted speech. Because of this reason, standard voice conversion models often struggle to reproduce accurate tonal contours, leading to robotic or broken audio. The Diffusion-SVC framework overcomes this linguistic challenge through a powerful generalization capability known as extrapolation.

Unlike older autoregressive models that generate audio sequentially and struggle with unseen data, this system processes the entire mel-spectrogram in a non-autoregressive parallel manner. More importantly, it achieves extrapolation by completely disentangling the linguistic content (extracted via ContentVec) from the fundamental frequency.

When an input Vietnamese audio contains a high-pitched tone (e.g., the Sharp tone) that falls completely outside the target speaker's natural vocal range in the training data, the system does not fail. By utilizing the Neural Source Filter (NSF) within the HiFi-GAN vocoder, the network mathematically generates a physical sine-wave excitation matching that exact high F0. This guarantees that the system can accurately infer and synthesize Vietnamese tonal variations, generalizing far beyond the acoustic boundaries of its original training dataset.

Aside from the tonal system, Vietnamese speech processing must also account for significant regional variations. The language is broadly categorized into Northern, Central, and Southern dialects, each with distinct phonological traits, consonant pronunciations, and tone merging behaviors [5].

In frameworks like Diffusion-SVC, the speech representations extracted from the source audio inherently capture the source speaker's specific regional pronunciation. Consequently, the converted speech retains the source speaker's dialect while adopting the target speaker's vocal timbre. This architectural trait introduces a unique challenge

in "cross-dialect" conversion: mapping the acoustic properties of a target speaker to a dialect they may not naturally speak. This mismatch can sometimes result in unnatural prosody or subtle phonetic artifacts, making it difficult to achieve perfect naturalness.

The research community has actively addressed these complexities through specialized evaluations and benchmarks. Notably, the VLSP 2025 task on Vietnamese Voice Conversion provides a standardized platform to evaluate the naturalness, speaker similarity, and intelligibility of converted Vietnamese speech across diverse scenarios [6]. The findings from these evaluations highlight that while modern architectures like diffusion models achieve high voice quality, resolving cross-dialect unnaturalness and maintaining perfect tonal accuracy remain primary objectives for future research.

1.7. Chapter summary

This chapter established the fundamental theoretical background necessary for understanding modern voice conversion systems. It began with an overview of speech processing and the evolution of voice conversion from early statistical methods to advanced deep learning approaches. Key acoustic representations, including the mel-spectrogram, fundamental frequency (F0), and linguistic content, were discussed as the essential building blocks for decoupling linguistic content from speaker identity.

Furthermore, the chapter introduced the core generative mechanisms underlying the frameworks, specifically DDPM and the shallow diffusion mechanism, which enable high-fidelity and computationally efficient voice synthesis. Finally, the specific characteristics and challenges of processing the Vietnamese language were analyzed, highlighting the critical need to preserve tonal accuracy and the complexities of handling regional dialects during the conversion process.

These foundational concepts provide the necessary context for the subsequent chapters. Building upon this theoretical base, the next chapter will delve into the detailed system architecture, the mechanism of feature disentanglement, and the specific methodology of the Diffusion-SVC framework.

CHAPTER 2: SYSTEM ARCHITECTURE AND METHODOLOGY

Purpose of the chapter

This chapter presents the detailed system architecture and methodology of the Diffusion-SVC framework[14]. Building upon the theoretical foundations established in Chapter 1, it explores the core modules of the system, including feature extraction, generative modeling, and waveform synthesis. A specific focus is placed on the feature disentanglement mechanism, which separates linguistic content from speaker identity, and the shallow diffusion process utilized for efficient inference. Furthermore, the chapter details the distinct pipelines for training and inference, showing how the system preserves the source speaker's regional dialect while successfully adopting the target speaker's vocal timbre.

Although this chapter does not provide an exhaustive review of all referenced models (which are discussed in their original publications), it establishes the foundation for the experimental setup and evaluations presented in subsequent chapters.

2.1. System Overview

The voice conversion system implemented in this project is built upon the open - source Diffusion-SVC framework [14]. This framework is an advanced, end-to-end generative system designed for high-fidelity voice conversion. The core objective of this architecture is to seamlessly transform the voice identity of a source speech signal into a target speaker's voice while strictly preserving the linguistic content, pronunciation, and intonation of the original input.

To achieve this complex transformation, the system pipeline is structured into three primary modules:

1. **Feature Extraction Module:** This module decomposes the input audio into independent components, specifically isolating the linguistic content, the fundamental frequency, and the speaker identity.
2. **Generative Acoustic Module (Diffusion Model):** Utilizing a shallow diffusion mechanism, this module acts as a bridge, combining the extracted content and pitch features with the target speaker's embedding to synthesize a high-quality mel-spectrogram.
3. **Waveform Synthesis Module (Vocoder):** The final stage employs an vocoder to convert the generated mel-spectrogram back into a listenable, natural-sounding audio waveform.

By decoupling the extraction of features from the generation process, the architecture ensures that the system can flexibly perform many-to-many voice conversions without suffering from speaker leakage or semantic distortion.

2.2. Feature Extraction and Disentanglement

The success of modern voice conversion relies heavily on the concept of feature disentanglement. The system must separate "what is being said" (linguistic content) from "who is speaking" (speaker identity). In the Diffusion-SVC framework, this is achieved by processing time-varying features and static features independently.

2.2.1. Linguistic Content Extraction

To capture the linguistic information of the speech signal, Diffusion-SVC utilizes ContentVec, a self-supervised speech representation model [11]. ContentVec processes the input audio frame-by-frame, outputting a sequence of time-varying vectors.

Crucially, ContentVec is explicitly trained to discard biometric speaker traits (such as vocal timbre and resonance) while retaining precise phonetic details. Because pronunciation, articulation, and regional dialects are fundamentally sequences of phonetic movements, they are entirely encoded within these units vectors. When processing an input signal, ContentVec acts as a phonetic mirror, faithfully extracting the exact pronunciation habits and dialect of the source speaker without attempting to normalize them. This ensures that the linguistic integrity of the source audio is fully preserved for the generation phase.

2.2.2. Fundamental Frequency and Volume Extraction

While ContentVec captures the phonetics, it does not retain pitch information. For tonal languages like Vietnamese, where pitch variations dictate semantic meaning, explicitly modeling the fundamental frequency is strictly required.

The framework employs robust pitch extraction algorithms (such as RMVPE, Harvest, or Dio) to track the F0 contour of the source audio. This extracted time-varying F0 sequence provides the necessary prosodic conditioning for the diffusion as well as the NSF-HiFiGan model, ensuring that the target voice accurately follows the melody, emotional inflection, and tonal variations of the original input.

The volume extraction process estimates the loudness of an audio signal by dividing the waveform into small frames and computing the RMS energy for each frame. This method provides a smooth representation of the audio amplitude over time. A threshold-based masking step is then applied to distinguish voiced regions from silent or low-energy segments, which helps improve the robustness of subsequent audio processing tasks.

2.2.3. Speaker Representation (Speaker Embedding)

Contrast to the dynamic, time-varying nature of ContentVec and F0 features, the identity of the target speaker is represented as a static, global vector known as a Speaker Embedding.

This embedding is a single, continuous vector applied uniformly across all time frames during the generation process. It encodes the anatomical characteristics of the target speaker's vocal tract, determining the overall timbre, warmth, and texture of the synthesized voice. Because this vector lacks a temporal dimension, it cannot store or impose any pronunciation habits or regional dialects. Consequently, the generative model is forced to rely entirely on the ContentVec features for articulation, resulting in the target speaker's voice naturally adopting the dialect and speaking style of the source input.

2.3. Feature Encoders and Acoustic(Naive) Model

2.3.1. Contentvec

ContentVec is built upon the HuBERT framework, utilizing a deep neural network to extract speech representations [11]. Its architecture consists of two main parts: a convolutional waveform encoder and a Transformer encoder.

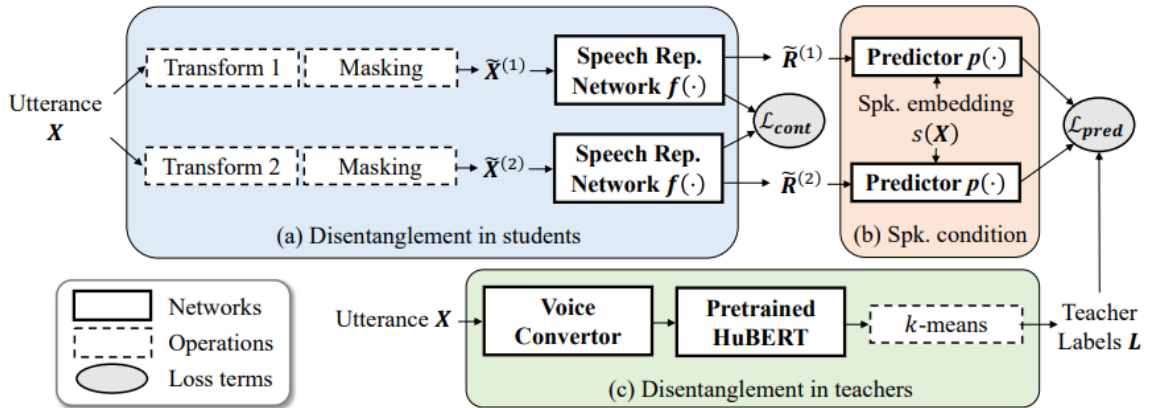


Figure 2.1: The overall structure of CONTENTVEC [11]

The input raw audio waveform, typically sampled at 16 kHz, is first processed by the Convolutional Feature Extractor. This encoder is constructed using 7 layers of 1D CNNs with a uniform channel dimension of 512. The precise layer configuration follows the sequence: $[(512, 10, 5)] + [(512, 3, 2)] * 4 + [(512, 2, 2)] * 2$, where each tuple defines the channels, kernel size, and stride, respectively. This architectural design yields a cumulative downsampling factor of 320 ($5 \times 2^4 \times 2^2$), which systematically downsamples the raw audio and extracts local acoustic features at a 50 Hz frame rate (equivalent to a 20 ms temporal stride). Since the final convolutional block outputs a

512-dimensional vector, a post-extraction linear projection layer is utilized to upscale the features into a 768-dimensional space, matching the required hidden dimension of the subsequent network.

These features are then fed into a multi-layer Transformer encoder. The model adopts a customized Base configuration comprising 12 Transformer layers with an embedding dimension of 768 and a Feed-Forward Network dimension of 3072, utilizing the GELU activation function.

Crucially, unlike the standard vanilla HuBERT framework, ContentVec introduces an explicit speaker conditioning mechanism within its encoder blocks. The network directly ingests a speaker embedding alongside the acoustic features, dedicating a specific subset of layers to process the speaker-associated information:

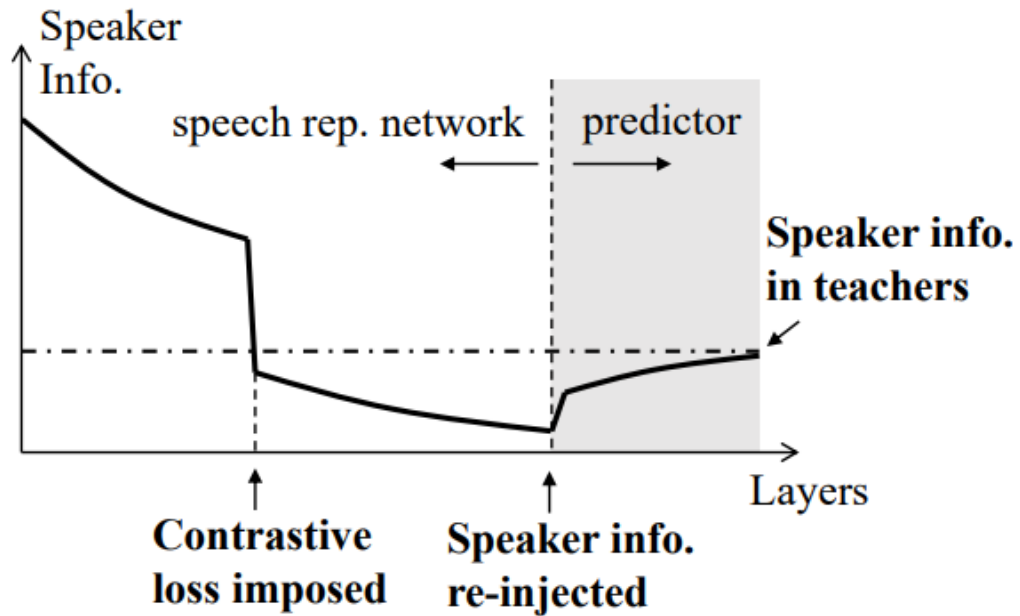


Figure 2.2: A conceptual curve of how speaker information changes through the network layers.[11]

This design actively enforces speaker disentanglement during the pre-training phase, allowing the self-attention blocks to capture long-range temporal dependencies and phonetic context while suppressing speaker-specific identities. The resulting representations primarily encode the pure linguistic content of the speech, providing a highly decoupled and robust input representation for downstream acoustic models and neural vocoders in voice conversion systems. This information will be helpful for the latter fine-tuning sections.

2.3.2. Auxiliary Acoustic Model (Naive Predictor)

The auxiliary acoustic model, implemented as the Unit2MelNaive module in the framework within the Diffusion-SVC repository [14], serves as the prior predictor for the shallow diffusion mechanism. Its primary function is to map the disentangled input features into a coarse, intermediate mel-spectrogram before the diffusion denoiser refines it. To achieve fast and accurate prior generation, the network is built upon a non-autoregressive, Transformer-based architecture.

The internal data flow of this naive predictor can be divided into several sequential processing blocks:

2.3.2.1. Content Feature Pre-processing (CNN Stack)

The units tokens extracted from ContentVec (denoted as units) are first passed through a 1D CNN stack. This stack utilizes Conv1d layers paired with GroupNorm and LeakyReLU activations to smooth the discrete units tokens and project them into a unified hidden dimension (typically 256 channels). This step provides a stable continuous representation of the phonetic content over time.

2.3.2.2. Acoustic Embeddings

Simultaneously, the continuous conditioning features must be embedded into the same dimensional space.

- **Fundamental Frequency:** To handle the wide dynamic range of human pitch, the raw F0 values are logarithmically scaled before being passed through a linear embedding layer. The transformation is defined as:

$$E_{f_0} = \text{Linear}(\ln(1 + \frac{f_0}{700}))$$

- **Volume/Energy:** The volume contour is similarly embedded via a linear layer to capture the dynamic loudness of the speech.
- **Speaker Identity:** The target speaker identity is processed either through a standard *nn.Embedding* lookup table (for fixed speaker IDs). The architecture also supports multi-speaker mixing by computing a weighted sum of multiple speaker embeddings.

2.3.2.3. Conditioning Fusion

Unlike concatenation-based architectures, this model employs element-wise addition to fuse the multi-modal features. The content representation $X_{content}$ is directly updated with the acoustic embeddings:

$$X_{fused} = X_{content} + E_{f_0} + E_{volume} + E_{spk}$$

This direct addition efficiently forces the network to modulate the phonetic structure with the desired pitch, loudness, and vocal timbre at every time step.

2.3.2.4. The PC_{mer} Decoder

The fused representation X_{fused} is fed into the core decoder, named PC_{mer} . This module is a variant of the Transformer Encoder, comprising multiple layers (defaulting to 3 layers) with 8-head multi-head self-attention mechanisms. The self-attention blocks allow the network to model long-range temporal dependencies, ensuring that the predicted coarse spectrogram maintains smooth and natural phonetic transitions.

2.3.2.5. Output Projection and Objective Function

Finally, the output from the PC_{mer} decoder passes through a *LayerNorm* and a weight-normalized linear layer to compress the 256-dimensional hidden state down to the target mel-spectrogram dimension (typically 128 mel-bins).

During the training phase, this network is optimized and trained independently from the diffusion model using a MSE loss between the predicted coarse spectrogram $\bar{x}_0$ and the ground-truth spectrogram x_0 . Additionally, an L_2 regularization term is applied to the convolutional weights to prevent overfitting:

$$L_{naive} = MSE(\bar{x}_0, x_0) + \alpha \Sigma W_{conv}^2$$

2.4. Generative Mechanism

The core generative power of the Diffusion-SVC framework lies in its denoising module. To fully comprehend the efficiency and high fidelity of the system, it is necessary to first examine the standard diffusion process and subsequently understand how the framework optimizes it using a shallow diffusion mechanism.

2.4.1. Denoising Diffusion Probabilistic Models

A standard DDPM operates through two parameterized Markov chains: the forward process and the reverse process [8].

The forward process gradually corrupts the original ground-truth mel-spectrogram x_0 by adding Gaussian noise over a predefined number of steps T (typically $T=1000$), controlled by a variance schedule β_t . The transition at a specific timestep t is defined as:

$$q(x_t|x_{t-1}) = N(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

A fundamental property of this process allows for the direct sampling of the noisy data x_t at any arbitrary timestep t directly from x_0 :

$$q(x_t|x_0) = N(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. At the final timestep T , x_T becomes nearly pure isotropic Gaussian noise.

The reverse process aims to reconstruct the original data x_0 starting from the pure noise x_T . The neural network, parameterized by θ , learns to estimate the transition probability and progressively remove the noise:

$$p_\theta(x_{t-1}|x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

While standard DDPMs generate exceptionally high-fidelity audio, starting the reverse process from pure noise requires hundreds of iterative denoising steps, making the inference phase computationally expensive and impractically slow for real-time applications.

2.4.2. Shallow Diffusion Solution

To overcome the inference bottleneck of standard DDPMs, the framework heavily adapts the Shallow Diffusion mechanism proposed in DiffSinger [9]. The core principle is to bypass the necessity of starting from pure random noise x_T by providing the diffusion model with a strong initial prior.

This is precisely where the auxiliary acoustic model (the Naive Predictor, detailed in Section 2.3.2) is utilized. The predictor generates a coarse mel-spectrogram $\widetilde{x}_0$ based on the disentangled phonetic content, pitch contour, and speaker embedding.

Instead of applying the full noise schedule, the shallow forward process only diffuses this coarse spectrogram up to a significantly smaller timestep k (where $k \ll T$, e.g., $k=100$):

$$x_k = \sqrt{\bar{\alpha}_k} \widetilde{x}_0 + \sqrt{1 - \bar{\alpha}_k} \epsilon$$

Consequently, the reverse denoising process only needs to commence from x_k and denoise for k steps to yield the final, refined mel-spectrogram x_0 . This approach bounds the computational cost while effectively correcting the acoustic artifacts and oversmoothed textures present in the naive prior $\widetilde{x}_0$.

***Note:** The Diffusion model used here also employs the same conditioning fusion as the Naive model (2.3.2.3) before parsing it along with the coarse mel to Wavenet denoiser.

2.5. Waveform Generation: NSF-HiFiGAN Vocoder

While the diffusion model successfully generates a high-fidelity mel-spectrogram, this output remains a two-dimensional time-frequency representation that cannot be directly rendered as an audio signal. To reconstruct the time-domain waveform from these acoustic features, the system employs a vocoder.

Specifically, Diffusion-SVC utilizes NSF-HiFiGAN, an advanced vocoder architecture that integrates two distinct methodologies to optimize synthesis quality [13], [14]. Fundamentally, HiFi-GAN operates as a GAN capable of high-speed, high-fidelity audio generation. However, conventional HiFi-GAN architectures exhibit vulnerabilities during voice conversion tasks, particularly pitch instability, phase glitches, and robotic artifacts when synthesizing out-of-domain fundamental frequencies.

To mitigate these pitch-related artifacts, the system incorporates the NSF mechanism. Rather than relying solely on the neural network to infer harmonic structures from the spectrogram, the NSF module utilizes the previously extracted F0 to explicitly generate a continuous sine-wave excitation signal. This physical excitation signal is directly injected into the HiFi-GAN generator network. By providing this explicit acoustic source, the network effectively models the human vocal tract functioning as a resonance filter. This approach ensures stable, artifact-free waveform synthesis, even when extrapolating to extreme pitch variations that fall outside the target speaker's natural vocal range.

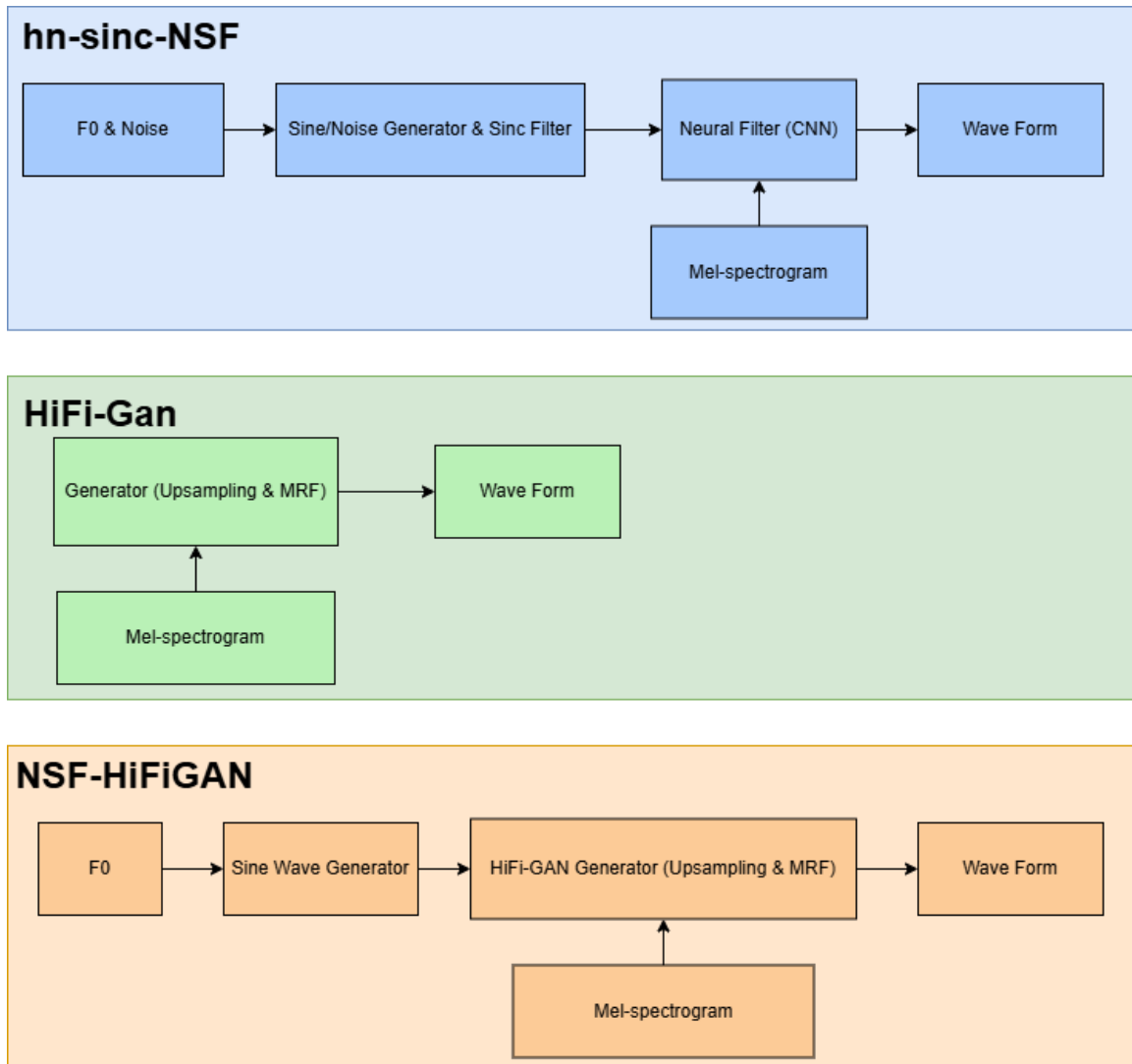


Figure 2.3: Pipeline overview of hn-sinc-NSF, HiFi-Gan and NSF-HiFiGan

2.6. Training and Inference Pipelines

The way data moves through the system completely changes depending on whether we are training the model or actually using it to convert a voice.

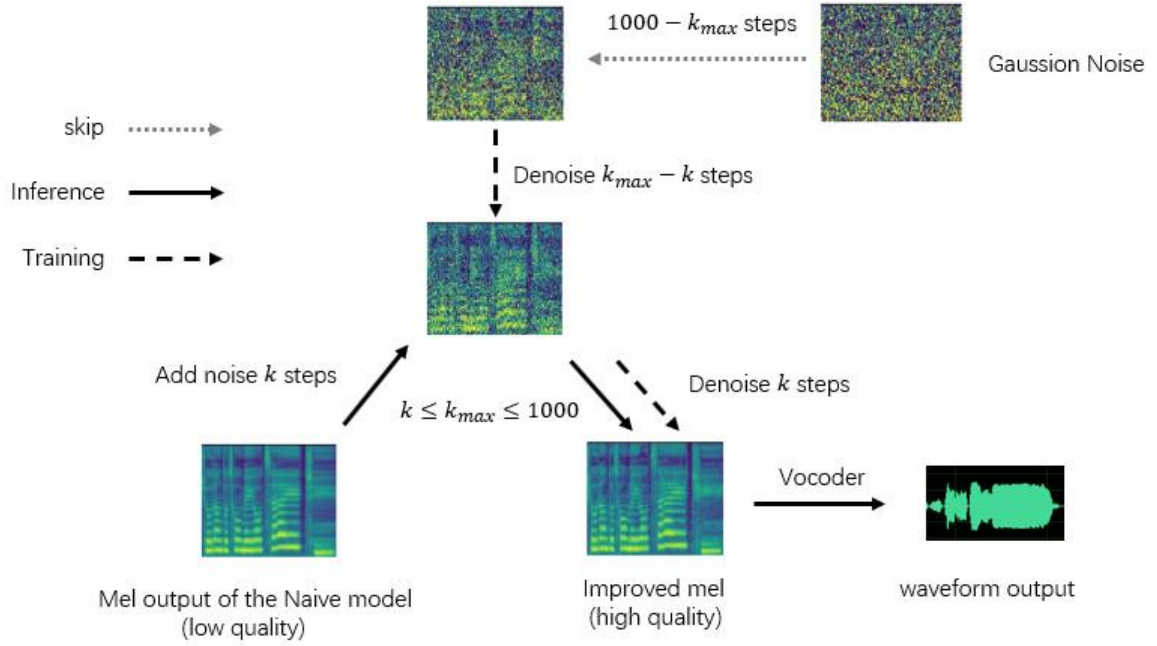


Figure 2.4: Overview of training and inference pipelines [14]

2.6.1. Training Pipeline

During the training phase, the system functions as an auto-encoder, optimizing for the reconstruction of the target speaker's acoustic features. The model is provided with audio recordings exclusively from the target speaker. The ContentVec model extracts the linguistic representations, which inherently encapsulate the target speaker's specific dialect and pronunciation patterns. Simultaneously, the fundamental frequency pitch contour is extracted.

2.6.1.1. Naive Acoustic Model Training Pipeline

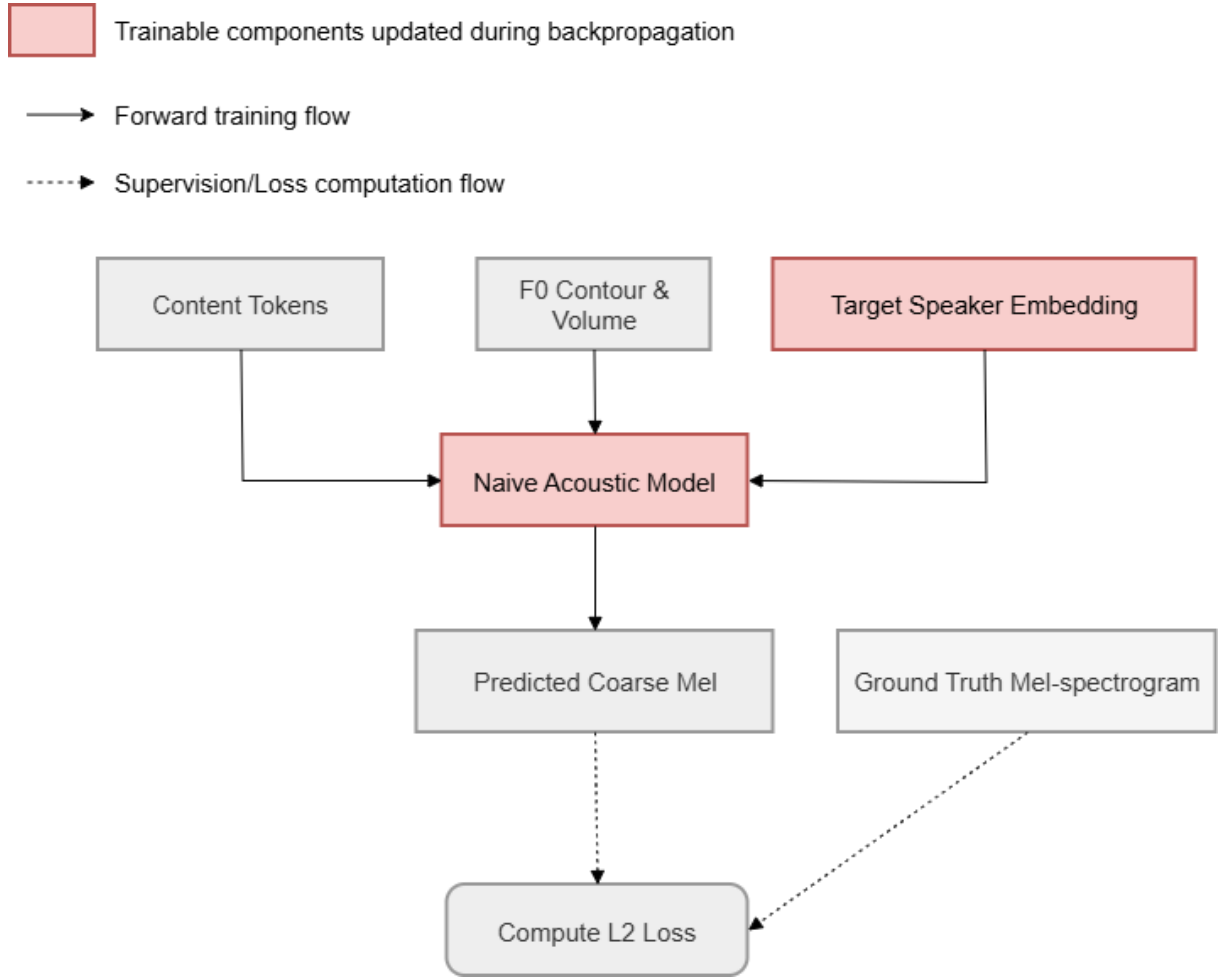


Figure 2.5: Naive model's training pipeline

The objective of this pipeline is to train the acoustic to map linguistic features to a coarse mel-spectrogram. This coarse mel-spectrogram serves as a prior for the subsequent Shallow Diffusion process.

- **Feature Extraction:** The target audio signal is passed through three parallel extraction blocks. The ContentVec algorithm extracts Content Tokens representing phonetic information. The RMVPE algorithm extracts the F0 contour representing pitch. Simultaneously, the STFT is employed to generate the Ground Truth Mel-spectrogram, which acts as the label for supervised learning.
- **Mel Prediction:** The Naive model receives three conditioning inputs: Content Tokens, the F0 contour, and the Target Speaker Embedding. Based on these inputs, the neural network synthesizes and predicts a Coarse Mel-spectrogram.

- Loss Computation: The predicted coarse mel and the ground truth mel are evaluated using the L2 loss function (MSE). This error is utilized for backpropagation to optimize the weights of the Naive model.

2.6.1.2. Diffusion Model Training Pipeline

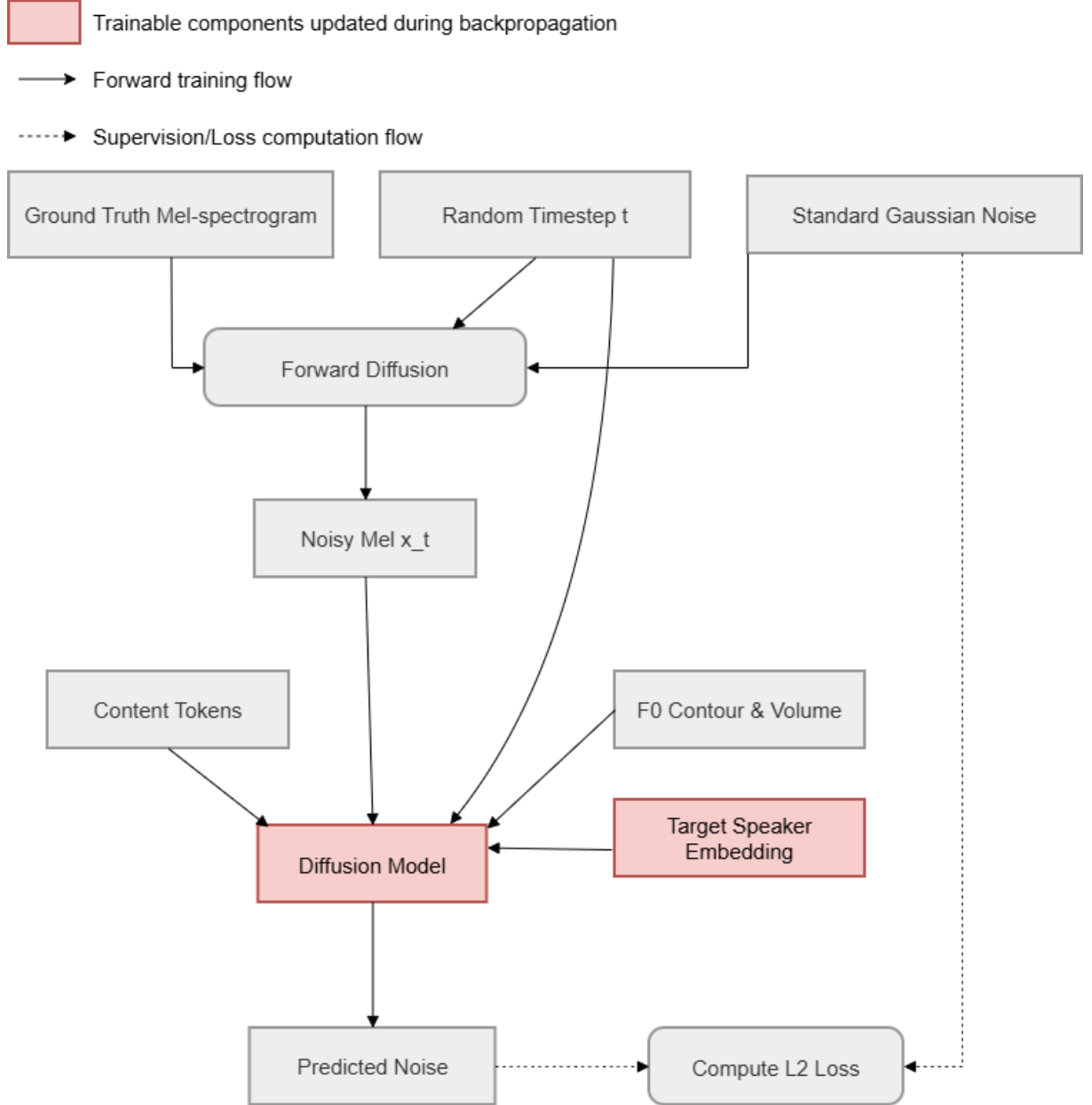


Figure 2.6: Diffusion model's training pipeline

Unlike the Naive model, which directly predicts the acoustic spectrum, the Diffusion model is trained to predict the amount of noise added to the signal.

- Forward Process: The q_{sample} block takes the ground truth mel (x_0) and a pure Gaussian noise matrix (ϵ) as inputs. The system randomly selects a timestep t between 0 and T . Based on t , the algorithm blends x_0 and ϵ according to a

closed-form mathematical ratio to produce the noisy mel-spectrogram at step t , denoted as $\mathbf{x}_t$.

- Denoising Process: The noisy spectrum $\mathbf{x}_t$ is passed through the Diffusion Block to predict the exact noise ϵ that was initially injected. To ensure the model does not denoise blindly, it is guided by four dashed-line conditioning signals: Time Embedding (to indicate the current noise variance level), Content Tokens, the F0 contour, and the Target Speaker Embedding. These signals continuously modulate the hidden states of the network.
- Optimization: The Predicted Noise and the Standard Gaussian Noise (True Noise) are compared via the L2 Loss function to compute the gradient and update the Diffusion model's calculations and WaveNet's weights. Randomly sampling t in each iteration enables the model to learn reverse diffusion across all noise levels.

The primary objective of the diffusion model is to utilize the target speaker's static embedding, the extracted content vectors, and the F0 contour to accurately reconstruct the original ground-truth mel-spectrogram. Through this optimization process, the model learns to map the target speaker's vocal timbre and linguistic habits back to their original audio representation. In this phase, both the linguistic content and the acoustic identity belong entirely to the target speaker.

2.6.2. The Inference Pipeline (Voice Conversion)

During the inference phase, the data flow is fundamentally modified to execute the voice conversion task. The system processes a recording from a distinct source speaker. ContentVec extracts the time-varying linguistic representations from the source audio, capturing the source speaker's specific pronunciation habits, pacing, and regional dialect.

The critical mechanism of voice conversion occurs at this stage: the source speaker's identity embedding is discarded and replaced with the target speaker's embedding. Because the target embedding is a static mathematical vector representing the physical characteristics of the vocal tract (such as timbre and resonance), it contains no phonetic or temporal information. Consequently, the diffusion model generates the audio conditioned on the target's timbre, but strictly guided by the pronunciation encoded in the source speaker's ContentVec sequence. This feature-swapping mechanism enables the system to produce an output that possesses the acoustic identity of the target speaker while seamlessly preserving the exact dialect, prosody, and phonetic nuances of the source speaker.

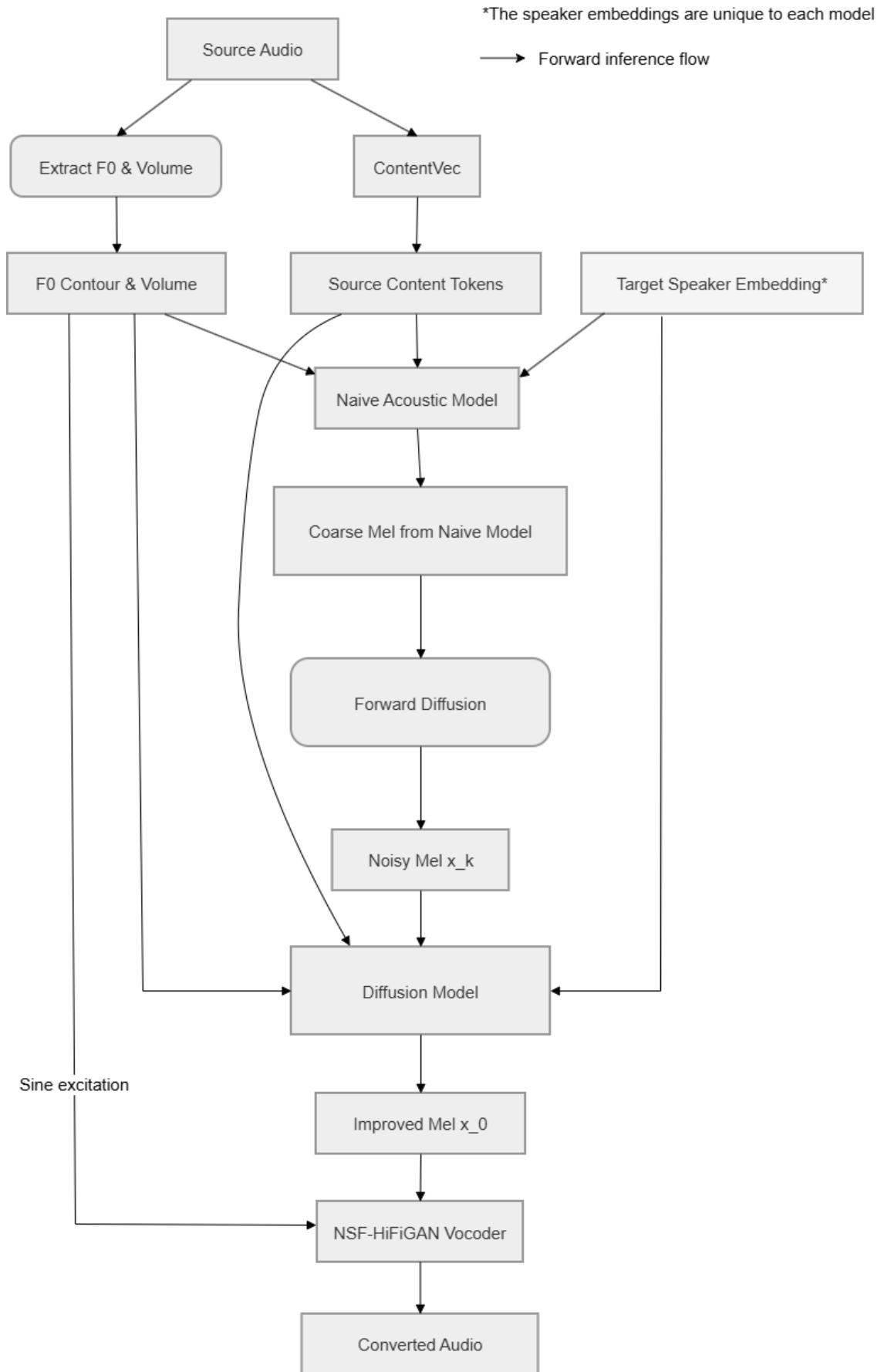


Figure 2.7: Inference pipeline

This diagram illustrates the operational flow when the system receives a source audio file and converts it to the target speaker's voice. This phase applies the Shallow Diffusion technique combined with the DPM-Solver acceleration algorithm.

- **Source Pre-processing:** The system extracts Content Tokens from the source audio. The fundamental frequency is also extracted.
- **Prior Generation:** Instead of starting from pure white noise (as in Full Diffusion), the system utilizes the Naive model to predict a Coarse Mel-spectrogram based on the source content, the pitch, and the target speaker embedding.
- **Shallow Diffusion:** The coarse mel prior is passed through the q_sample function to be corrupted by noise up to a specific step k , generating x_k . This semi-noisy state provides a highly accurate initialization space for the diffusion model. The WaveNet, coupled with the DPM-Solver sampling algorithm, performs reverse denoising steps from k to 0 , generating a high-fidelity mel-spectrogram (x_0).
- **Vocoding:** The refined mel-spectrogram is forwarded to the NSF-HiFiGAN vocoder, which re-use the F0 contour to generate a physical sine-wave excitation signal. This sine wave acts as a NSF, assisting the generator network in synthesizing the final waveform with pitch stability. This entirely overcomes phase glitches and distortion artifacts, especially in out-of-domain frequencies. The final output is the converted voice featuring the target timbre while precisely retaining the original speaker's prosody and linguistic content.

2.7. Baseline models

The authors of the Diffusion-SVC repository provide a trained dual-speakers model [14], both of which will be used in this project to establish baseline performance metrics. Both models utilize pretrained ContentVec (specifically the contentvec768l12 variant - The same one is used for finetuning in this project).

1. Naive model

The Naive model is configured with the following parameters:

- **Unit Encoder:** contentvec768l12
- **Training Duration:** 200,000 steps without pretrain model.
- **Dataset:** Dual-speaker corpus consisting of opencpop (Chinese female singing) and kiritan (Japanese female speech/singing).

2. Diffusion model

The Diffusion model's parameters include:

- Unit Encoder: contentvec768l12
- K_step_max: 100. (Maximum diffusion steps)
- Training Duration: 600,000 steps without pretrain model.
- Dataset: Identical dual-speaker corpus (opencpop and kiritan).

Evaluating these configurations against the fine-tuned models is essential for assessing their practical applicability and understanding the extent to which pretrained Shallow Diffusion models require additional fine-tuning on Vietnamese datasets. This evaluation also provides insight into how effectively the original models can adapt to the linguistic complexity, tonal characteristics, and phonetic nuances of the Vietnamese language. To comprehensively quantify this, the models will be assessed using WER to evaluate the accuracy of converted speeches and NISQA scores to gauge perceived naturalness, all of which will be discussed in Chapter 3. Furthermore, comparing the performance across different configurations helps identify the trade-offs between computational efficiency, synthesis quality, and language adaptation capability, thereby offering a clearer perspective on the most suitable approaches for Vietnamese speech generation tasks.

2.8. Chapter Summary

This chapter detailed the architectural components and theoretical foundations of the Diffusion-SVC framework. The analysis demonstrated how the system effectively disentangles linguistic content and dialect, extracted via ContentVec, from the static vocal timbre represented by the speaker embedding. Furthermore, the generative mechanism was examined, highlighting how the shallow diffusion approach accelerates the denoising process by utilizing an acoustic prior without compromising high-fidelity audio synthesis.

The chapter also discussed the integration of the NSF-HiFiGAN vocoder, for stable, artifact-free waveform generation from the predicted mel-spectrograms. Finally, a comparison between the training and inference pipelines illustrated the feature-swapping mechanism responsible for the system's cross-dialect conversion capabilities. With the theoretical and architectural blueprint established, the subsequent chapter will present the experimental setup, detailing the datasets, implementation configurations, and evaluation metrics used to assess the system's empirical performance.

CHAPTER 3: EXPERIMENTS AND EVALUATION

Purpose of the chapter

This chapter details the experimental setup, training procedures, and evaluation metrics for the Diffusion-SVC model applied to the Vietnamese language. The discussion consists of dataset selection, audio preprocessing, hardware configurations, fine-tuning models and hyperparameter tuning. The chapter also presents the evaluation results to demonstrate the system's capability in multi-gender voice conversion and cross-dialect preservation.

3.1. Experimental Setup and Datasets

To ensure the reproducibility of the results and to provide a clear context for the computational demands of the system, this section details the hardware configurations, software environments, and the specific speech corpora utilized during the fine-tuning and evaluation phases.

3.1.1. Hardware and Software Environment

Training deep generative models, particularly diffusion-based, Contentvec and NSF-HiFiGAN vocoder, requires substantial computational resources. All experiments, including data preprocessing, feature extraction, and model fine-tuning, were conducted on the specific hardware and software configurations summarized below:

Hardware Configuration:

- Central Processing Unit (CPU): AMD Ryzen 5 5600X
- Graphics Processing Unit (GPU): NVIDIA GeForce RTX 3070 with 8GB of VRAM.
- Random Access Memory (RAM): 32GB DDR4
- Storage: 1TB SSD.

Software Environment:

- Operating System: Windows 11 Pro 25H2
- Programming Language:
 - Contentvec: Python 3.7
 - NSF-HiFiGan, Diffusion, Utilities: Python 3.10
- Deep Learning Framework and Compute Platform:

The efficacy of the Diffusion-SVC framework heavily relies on the acoustic quality and phonetic diversity of the training data. To adapt the pretrained models to Vietnamese phonology and the specific vocal characteristics of the target speaker, this project utilized two distinct Vietnamese speech corpora: the VieNeu dataset and the infore1 dataset:

-
- The bar chart displays the frequency of each grade. The y-axis represents the 'Count' of notes, ranging from 0 to 80,000. The x-axis represents the 'Notes' (grades). Each bar is labeled with its percentage of the total.
- | Notes | Count (approx.) | Percentage |
|-------|-----------------|------------|
| D | 100 | 0.00% |
| E | 100 | 0.00% |
| F | 100 | 0.00% |
| G | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00% |
| B- | 100 | 0.00% |
| C | 100 | 0.00% |
| C- | 100 | 0.00% |
| D | 100 | 0.00% |
| D+ | 100 | 0.00% |
| E | 100 | 0.00% |
| E+ | 100 | 0.00% |
| F | 100 | 0.00% |
| F+ | 100 | 0.00% |
| G | 100 | 0.00% |
| G+ | 100 | 0.00% |
| A | 100 | 0.00% |
| A+ | 100 | 0.00% |
| B | 100 | 0.00 |

Figure 3.1: Notes distribution of male subset

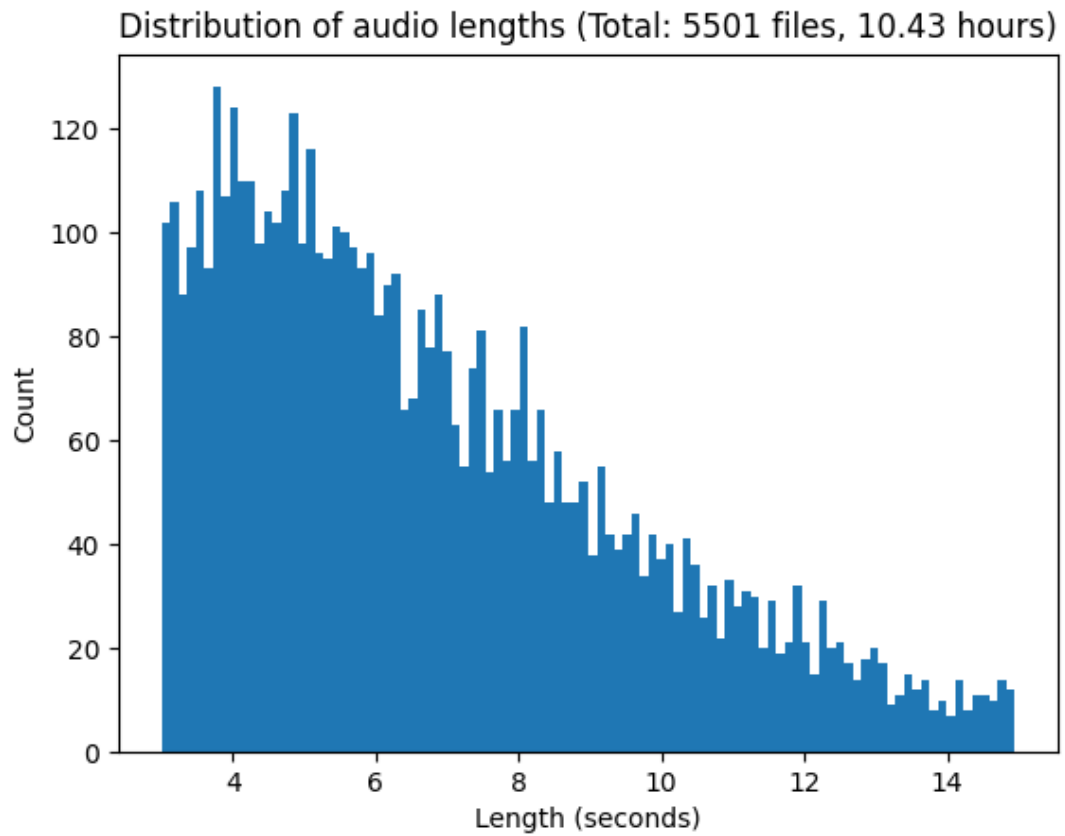


Figure 3.2: Length distribution of male subset

- Female Voice Dataset: A subset was extracted from the infore1_25hours dataset [4]. A total of 10 hours of standard female speech was selected:

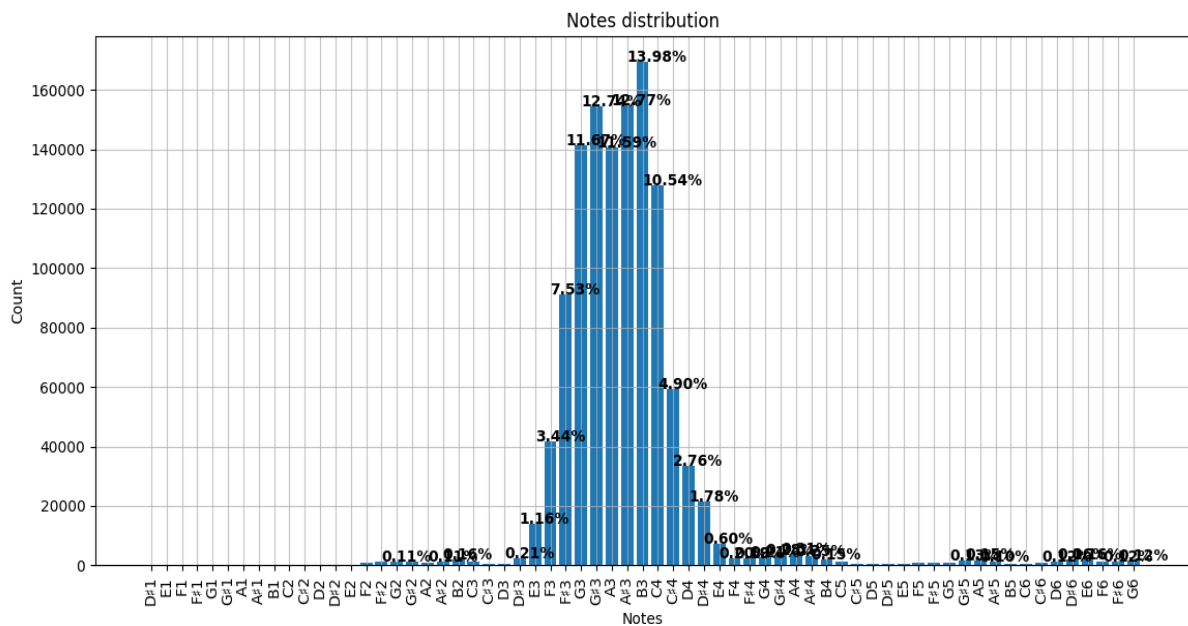


Figure 3.3: Notes distribution of female subset

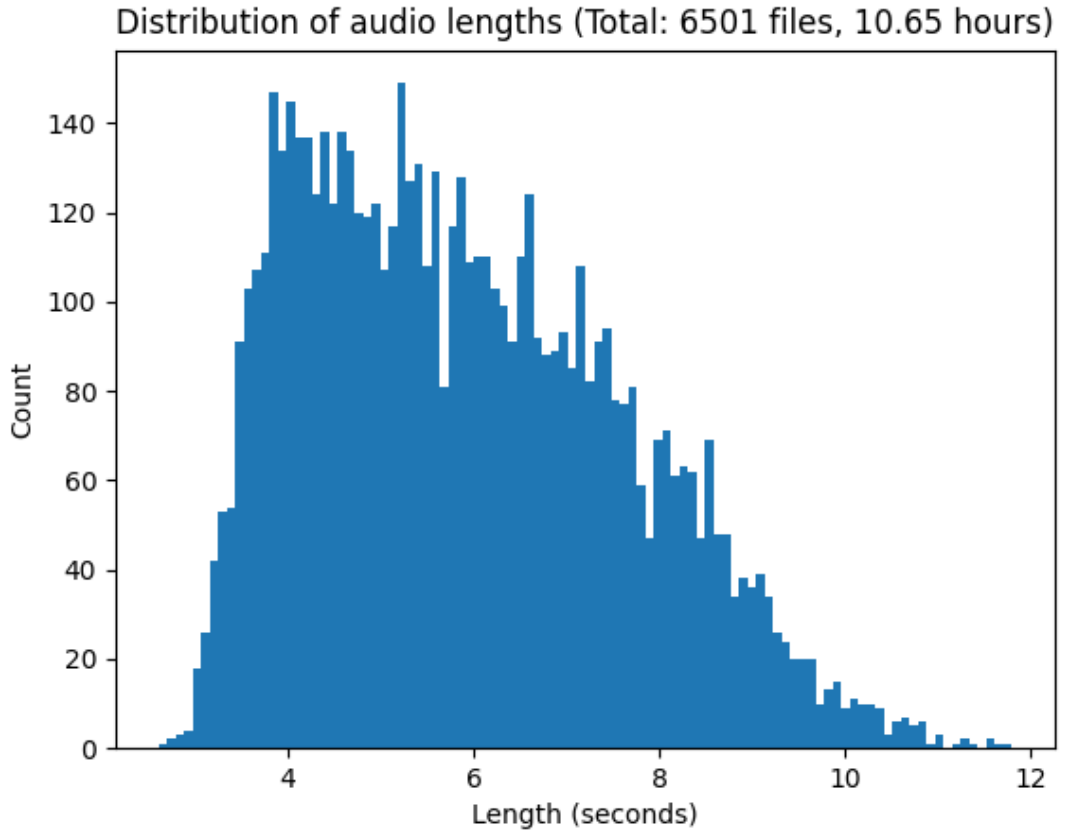


Figure 3.4: Length distribution of female subset

All extracted audio files have been verified to ensure that none are empty and that each file is in .wav format with as little noise and background silence as possible.

3.2. Data Preprocessing

The proposed voice conversion framework is not a monolithic codebase but rather an integration of three independent open-source projects: the ContentVec phonetic extractor, the core Diffusion-SVC system (encompassing the Naive and Diffusion models), and the NSF-HiFiGAN vocoder. Because these repositories operate in isolated environments with distinct DataLoader implementations, raw audio cannot be processed centrally. Instead, the data preprocessing pipeline is inherently decoupled into three standalone workflows, each tailored to satisfy the strict input formatting requirements of its respective project.

3.2.1. Contentvec

The ContentVec repository operates independently to learn speaker-disentangled phonetic representations. Its training and inference pipelines require the audio corpus to be formatted according to its specific acoustic standards before any feature extraction can occur.

- **Acoustic Standardization:** All raw audio files are strictly converted to a single-channel (mono) format and resampled to a target rate of 16 kHz using a high-fidelity anti-aliasing filter. This specific sampling rate is a hard constraint for the pretrained ContentVec architecture.
- **Corpus Structuring:** The normalized audio chunks are organized into isolated directory structures as mandated by the project's data ingestion scripts. Extended periods of silence are trimmed using Voice Activity Detection (VAD) to ensure the model focuses exclusively on phonetic transitions rather than background noise.

Since the original repo only provides a simple `setup.sh` for the training params, an improvised preprocessing pipeline had been made for the model on Windows based on Hubert document, on which Contentvec's code is also based. Because the model must learn phonetic representations without explicit human annotations, the preprocessing pipeline is extensive, requiring the generation of discrete "pseudo-labels" through K-Means clustering. The entire data preparation workflow, executed utilizing the fairseq toolkit with integrated Contentvec files, is delineated into five sequential stages:

- **Stage 1: Manifest Generation and Dataset Splitting:** The initial phase involves indexing the audio corpus. The `wav2vec_manifest.py` script scans the standardized 16 kHz audio directory and constructs a manifest file (TSV format) containing the absolute paths and frame lengths of every audio file. During this step, the dataset is deterministically partitioned, with 10% of the data allocated to the validation set (`--valid-percent 0.1`) and the remaining 90% reserved for training.

```

1  D:/Tools/Projects/contentvec/dataset/wavs
2  nam/0000.wav      98337
3  nam/0002.wav      62161
4  nam/0003.wav      97120
5  nam/0004.wav      89409
6  nam/0005.wav      93217
7  nam/0006.wav      85505
8  nam/0008.wav      67601
9  nam/0010.wav      85825
10 nam/0011.wav      180657

```

Figure 3.5: First few entries of train.tsv (The same for valid.tsv)

- **Stage 2: Continuous Feature Extraction (Dumping Features):** Before clustering, the system must extract dense, continuous representations from the audio. The `dump_hubert_feature.py` script leverages a pretrained base model checkpoint (`checkpoint_last.pt`). In this implementation, the features are explicitly extracted

from the 12th Transformer encoder layer (layer=12). This extraction is executed twice: once for the train split (processing 10,840 utterances) and once for the valid split (processing 1,162 utterances). The output of this stage is a high-dimensional feature matrix (e.g., shape [3423896, 768]) cached to the local storage.

Name	Date modified	Type	Size
 train_0_1.len	16/3/2026 21:40	LEN File	53 KB
 train_0_1.npy	16/3/2026 21:40	NPY File	10,271,689 ...
 valid_0_1.len	16/3/2026 21:46	LEN File	6 KB
 valid_0_1.npy	16/3/2026 21:46	NPY File	1,087,537 KB

Figure 3.6: Cached file prepared for clustering

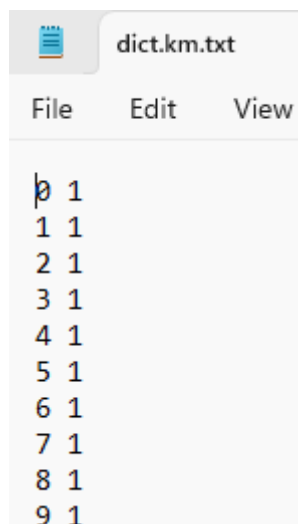
- Stage 3: K-Means Clustering for Discretization: To generate the discrete targets required for the masked prediction task, the continuous features undergo K-Means clustering using the `learn_kmeans.py` script.
 - The algorithm initializes via the `k-means++` method and is configured to partition the acoustic space into 500 distinct phonetic clusters (`--n_clusters=500`).
 - The algorithm processes the 3.42 million frames through a mini-batch approach (`batch_size=10000`). As observed in the training logs, the cluster inertia progressively minimizes, eventually converging (halting improvements) at step 1,143, indicating the successful mapping of the continuous acoustic space into a finite set of 500 discrete acoustic units. The resulting cluster centroids are serialized into a `.km` model file.

```
Minibatch step 1141/34238: mean batch inertia: 34.7037625, ewa inertia: 34.63784316907791
Minibatch step 1142/34238: mean batch inertia: 34.551625, ewa inertia: 34.63733954312726
Minibatch step 1143/34238: mean batch inertia: 34.628275, ewa inertia: 34.637286594436766
Converged (lack of improvement in inertia) at step 1143/34238
2026-03-16 22:00:28 | INFO | learn_kmeans | total inertia: 34.63038
2026-03-16 22:00:28 | INFO | learn_kmeans | finished successfully
```

Figure 3.7: Kmean algorithm found the convergence at step 1143

- Stage 4: Pseudo-Label Assignment: With the K-Means dictionary established, the `dump_km_label.py` script is executed across both the training and validation feature sets. This script assigns every single audio frame to its nearest K-Means cluster centroid. These cluster IDs (integers from 0 to 499) act as the discrete pseudo-labels (targets) that the ContentVec model will attempt to predict during its fine-tuning phase.

- Stage 5: Speaker Embedding Generation (spk2info dictionary): A defining characteristic of ContentVec is its objective to disentangle phonetic content from speaker identity. To achieve this, the model requires explicit speaker information during training. A custom script (create_contentvec_dict.py) is executed to pass the audio files through a pretrained Voice Encoder. This process extracts generalized speaker embeddings (e.g., separating male and female timbres across subsets of 150 to 200 samples) and compiles them into a structured spk2info.dict file. This dictionary allows the ContentVec loss function to actively penalize the network if it encodes speaker-specific traits into the phonetic tokens.
- Stage 6: Dummy dictionary in labels (dict.km.txt): The dict.km.txt is a mandatory metadata file that acts as the model's acoustic vocabulary. It maps the 500 discrete units generated by the K-means algorithm to the model's output neurons. The format [index] [count] is a requirement of the Fairseq Dictionary Class. While the first column represents the 500 discrete acoustic units (0-499), the second column is assigned a constant value of 1. This serves as a uniform frequency weight, ensuring that the model initializes its embedding layers without statistical bias toward any specific acoustic cluster, thereby maintaining a balanced learning objective across the entire phonetic space. Since this is just a dummy dictionary file, it can be generated by a simple script that writes the first column from 0 to 500 (or 100 for 100 kmean clusters) and fills the second column with 1s.



File	Edit	View
0	1	
1	1	
2	1	
3	1	
4	1	
5	1	
6	1	
7	1	
8	1	
9	1	

Figure 3.8: First few entries of the dummy dictionary

This is how the dataset folder looks after preprocessing data:

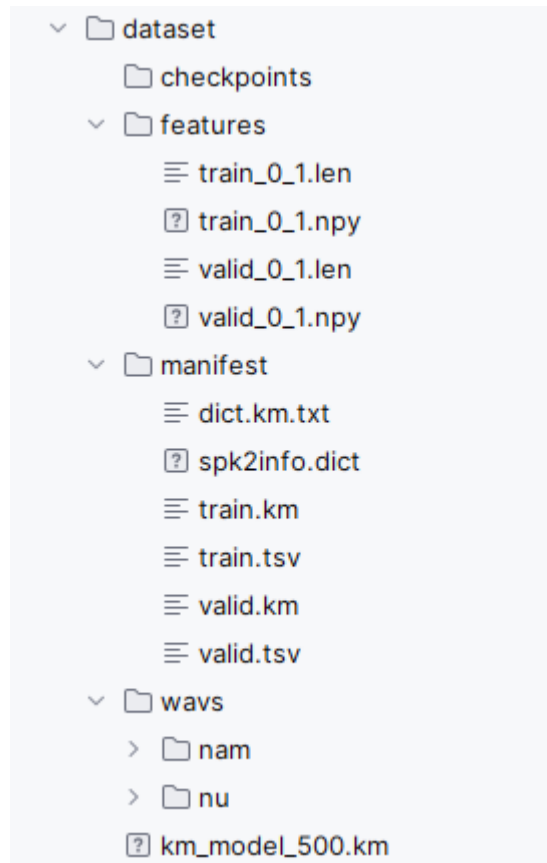


Figure 3.9: Contentvec dataset folder structure after the data is preprocessed

3.2.2. Diffusion

The Diffusion repository[14] provides a preprocess script that can be run from console, making preprocessing data much easier than Contentvec. The dataset, which is resampled to 44.1khz, set up process can be done in two ways:

- Manual configuration:
 - Place all training set data (.wav format audio clips) in the data/train/audio folder, or in a directory specified in the configuration file such as xxxx/yyyy/audio.
 - Place all validation set data (.wav format audio clips) in the data/val/audio folder, or in a directory specified in the configuration file such as aaaa/bbbb/audio.
- Random Selection:
 - Run python draw.py. The program will help selecting validation set data (parameters such as the number of files to be extracted in draw.py can be modified).

The data folder structure must be the same as follow:

```

data/
├── train/
│   ├── audio/
│   │   ├── 1/
│   │   │   ├── aaa.wav
│   │   │   └── bbb.wav
│   │   └── 2/
│   │       ├── ccc.wav
│   │       └── ddd.wav
└── val/
    ├── audio/
    │   ├── 1/
    │   │   └── eee.wav
    │   └── 2/
    │       └── fff.wav

```

After modifying the config file, the preprocessing process can be started by running ***python preprocess.py -c configs/config.yaml*** which will create 6 folders and 1 npy file:

- **aug_mel**: A random pitch shift (ranging from -5 to +5 semitones) is applied to the audio signal. The script consequently extracts an augmented mel-spectrogram representing this transposed pitch.
- **aug_vol**: The maximum amplitude of the audio tensor is computed, and a logarithmic volume shift (`log10_vol_shift`) is randomly sampled. The raw audio is multiplied by this scalar before generating the augmented volume.
- **f0**: The fundamental frequency is extracted using a high-precision pitch tracker (e.g., RMVPE or Parselmouth), configured with boundaries appropriate for human speech.
- **mel**: The acoustic targets for the network are generated using a defined Vocoder class. To ensure perfect alignment during the final waveform synthesis phase, the mel-spectrogram extraction rigidly adheres to the vocoder's internal sampling rate and hop size parameters.
- **units**: This module integrates the phonetic extractor (such as the previously fine-tuned ContentVec/HuBERT model), transforms the time-domain audio into a

sequence of discrete linguistic representations (units) aligned with that encoder's specific hop_size.

- volume: A volume extractor computes the frame-level energy/amplitude of the signal, the volume contour acts as an auxiliary conditioning parameter, enabling the diffusion model to replicate the natural dynamic range and breathing pauses of the target speaker.
- pitch_aug_dict.npy: A registry used to track and serialize the shift values for each file, ensuring synchronization between the conditioning signals and the augmented targets during training.

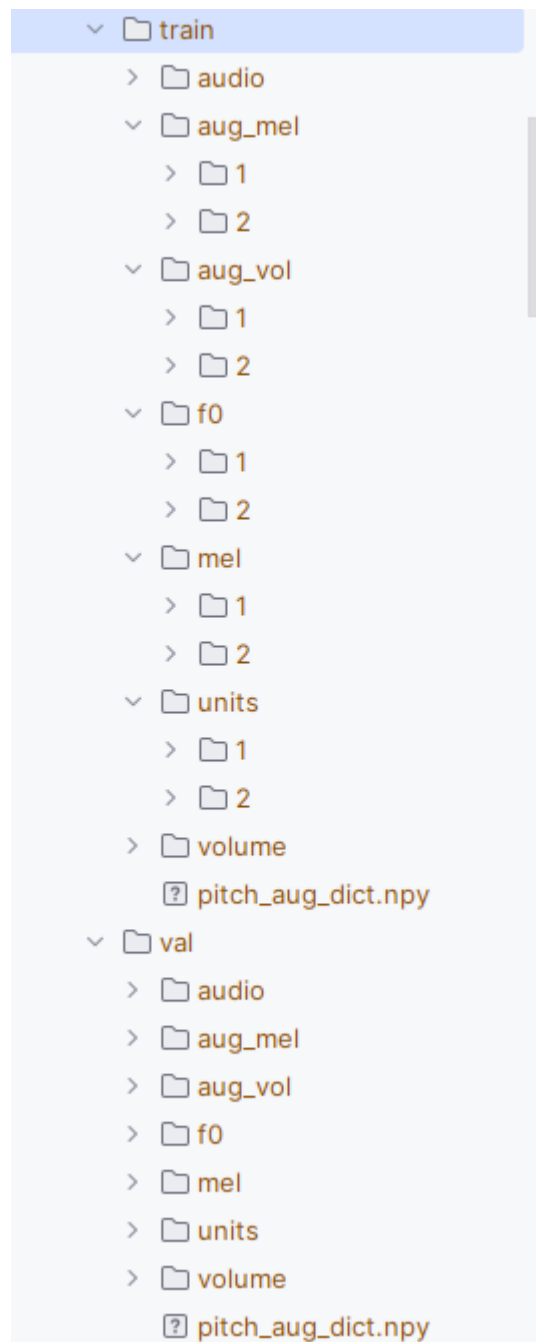


Figure 3.10: Diffusion dataset folder structure after the data is preprocessed

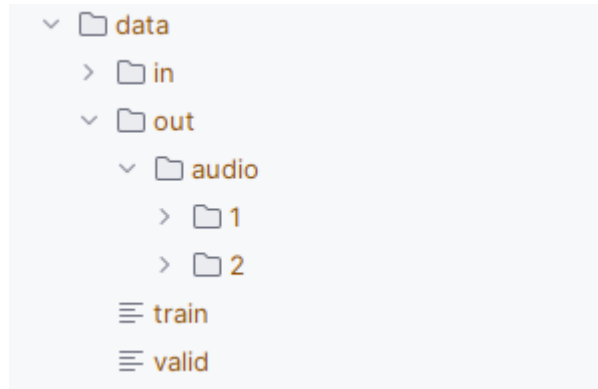
(All the preprocessed file will have .npy format.)

3.2.3. NSF-HiFiGan

The NSF-HiFiGan vocoder used in this project can be found on SingingVocoders repository [15] along with it's preprocess scripts. The process is simple:

- Resampling the audios to 44.1khz.
- Modify the `data_input_path`, `data_out_path` and `val_num` parameters inside the config file, and then run the provided preprocess command: ***python process.py -config (config path) --num_cpu (Number of cpu threads used in preprocessing) --strx (1 for a forced absolute path 0 for a relative path)***.

After that the data folder's structure should be the same as the 3.7 figure:



(All the preprocessed file will have .npz format.)

3.3. Fine-tuning Process and Evaluation

This section presents the empirical optimization of the Diffusion-SVC framework. Instead of providing isolated training logs, the system's evolution is described through an iterative refinement strategy. In each phase, specific algorithmic or hyperparameter modifications are introduced, followed by both objective and subjective evaluations to assess their impact. Since all three projects are integrated into a single pipeline, all models are trained, fine-tuned, and experimented with under different configurations.

3.3.1. Evaluation methods

To comprehensively evaluate the performance of the proposed models, this project employs a combination of objective metrics and automated perceptual assessment models. The evaluation is divided into metrics requiring a time-aligned ground-truth reference (used for reconstruction tasks) and non-intrusive metrics (used for overall quality assessment regardless of speaker identity).

- **Reference-based Metrics (For Reconstruction Evaluation)**

These metrics require the synthesized audio and the ground-truth audio to be perfectly time-aligned and spoken by the same speaker. They are utilized to measure the physical and perceptual fidelity of the feature extractors (Encoders) and acoustic models.

1. Spectral Convergence (SC) [16]

Spectral Convergence measures the relative difference between the magnitude spectrograms of the predicted and ground-truth signals using the Frobenius norm. A lower SC value indicates that the model accurately reconstructs the energy distribution across frequency bands, thereby minimizing spectral distortion.

$$SC = \frac{\| |STFT(x)| - |STFT(\hat{x})| \|_F}{\| |STFT(x)| \|_F}$$

Where x is the ground-truth signal and $\hat{x}$ is the synthesized signal, $\|\cdot\|_F$ is the Frobenius norm over time and frequency.

2. Log STFT Magnitude Loss [16]

Unlike SC, which operates on a linear scale, this metric evaluates the reconstruction error on a logarithmic scale using the L^1 norm. The logarithmic scale closely simulates how the human auditory system perceives differences in sound energy. A lower loss indicates that fine spectral details, including high-frequency harmonics, are well-preserved.

$$L_{log} = \| \log(|STFT(x)| + \epsilon) - \log(|STFT(\hat{x})| + \epsilon) \|_1$$

where $\|\cdot\|_1$ is the L^1 norm and ϵ is a small number.

3. Mel-Cepstral Distance (MCD) [17]

MCD is a standard metric used to evaluate timbre distortion. It calculates the Euclidean distance between the Mel-Frequency Cepstral Coefficients (MFCCs) of the ground-truth and synthesized audio. In voice synthesis, MCD reflects the degree of timbre leakage and spectral envelope accuracy. A lower MCD score indicates higher spectral similarity and better timbre purity.

$$MCD = \frac{10}{\ln 10} \sqrt{2 \sum_{k=1}^K (c_k - \hat{c}_k)^2}$$

Where c_k and $\hat{c}_k$ represent the k -th MFCC of the ground-truth and the synthesized signals, respectively, and k is the total number of coefficients.

4. F0 Root Mean Square Error (F0 RMSE)

F0 is the fundamental element determining melody and intonation. F0 RMSE measures the root mean square deviation between the F0 contours of the original and synthesized audio. The frequency in Hertz is converted to Cents (100 Cents = 1 musical semitone) before calculation to align linearly with human pitch perception. A lower F0 RMSE confirms that the model accurately follows the target melody without pitch jumps.

$$RMSE_{cents} = \sqrt{\frac{1}{T} \sum_{t=1}^T (p_t - \hat{p}_t)^2}$$

Where $p_t = 1200 \log_2(F0_t/10)$, representing the pitch value in cents at frame t .

5. PESQ (Perceptual Evaluation of Speech Quality) [18] & STOI (Short-Time Objective Intelligibility) [19]

- PESQ: This metric assesses audio quality based on a psychoacoustic model simulating human perception. Wideband PESQ scores range from -0.5 to 4.5. A higher score indicates natural, smooth audio free of irritating artifacts.
- STOI: This metric evaluates speech intelligibility (the listener's ability to correctly identify spoken words). Scores range from 0.0 to 1.0. A score approaching 1.0 demonstrates that the phonetic information is well-preserved, resulting in clear and articulate speech.

• Non-Intrusive Speech Quality Assessment (NISQA) [20]

While reference-based metrics are highly effective for same-speaker reconstruction, they are not applicable for cross-lingual or cross-speaker Voice Conversion evaluations due to time and phase misalignment. To address this, NISQA, a deep learning-based model designed for non-intrusive (reference-free) speech quality prediction, is used. To evaluate voice conversion quality, the NISQA TTS variant is used, hereafter referred to as NISQA. It is specifically trained to predict the naturalness of synthetic speech generated by voice conversion and text-to-speech systems.

NISQA evaluates the physical acoustic quality and identifies AI-generated artifacts without requiring a ground-truth reference, making it independent of language barriers or speaker identities. The model predicts a Mean Opinion Score (MOS) on a scale from

1.0 to 5.0, where higher values correspond to more natural and human-like speech quality.

- **Word Error Rate (WER)**

To evaluate the intelligibility and preservation of linguistic content in the synthesized speech, Word Error Rate is employed. In Voice Conversion, an effective model must alter the speaker identity while leaving the underlying spoken words completely intact. WER quantifies this capability by measuring the operational discrepancy between the transcribed text of the converted audio and the original ground-truth reference, strictly accounting for word substitutions, deletions, and insertions.

An Automatic Speech Recognition model is employed, specifically OpenAI Whisper (large-v3), to act as an objective, independent listener that transcribes the generated audio outputs. To ensure the metric strictly evaluates linguistic accuracy rather than superficial formatting mismatches, a text normalization protocol is applied to both the ground-truth references and the ASR transcriptions to standardize them into lowercase text void of punctuation. The resulting corpus-level WER serves as a direct indicator of the model's performance, where a lower error rate denotes superior phonetic accuracy and successful content preservation.

- **Speaker Similarity**

To evaluate the degree to which speaker identity is preserved in synthesized speech, the Speaker Similarity metric is employed as a key evaluation measure. In Voice Conversion, an effective model must not only maintain the linguistic content of the utterance but also transform the speaker's identity such that the generated speech closely resembles the target speaker. Speaker Similarity quantifies this capability by comparing speaker-related representations extracted from the converted audio and a reference utterance of the target speaker.

A pre-trained speaker verification model from SpeechBrain[21] is utilized, specifically the ECAPA-TDNN model (spkrec-ecapa-voxceleb), to extract discriminative speaker embeddings from audio signals. This model, trained on the VoxCeleb dataset, maps each utterance into a fixed-dimensional embedding space that encodes speaker identity information in a highly separable manner.

3.3.2. Fine-tuning pipeline

3.3.2.1. Baseline (Zero-Shot Configuration)

Configuration: Utilizing the default pretrained ContentVec, NSF-HiFiGan, Naive, and Diffusion models without domain-specific hyperparameter tuning or source code interventions.

Evaluation: Since the pretrained diffusion model use different target speakers from the datasets used in this project (opencpop and kiritan), NISQA and WER metrics were measured across 100 audio files from the *dolly-audio-1000h-vietnamese* dataset [22], which was used solely for evaluation purposes in this project.

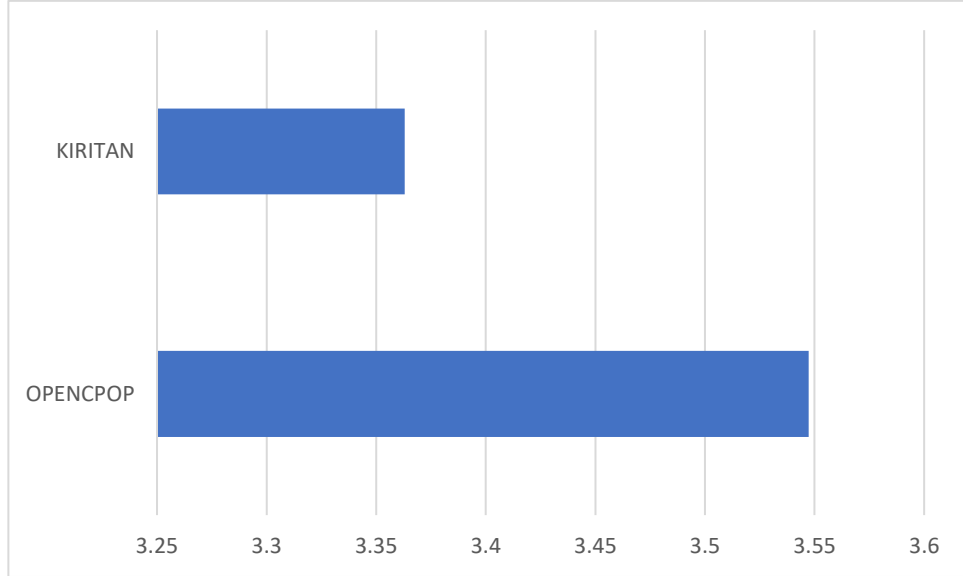


Figure 3.11: Baseline NISQA scores (Averaged)

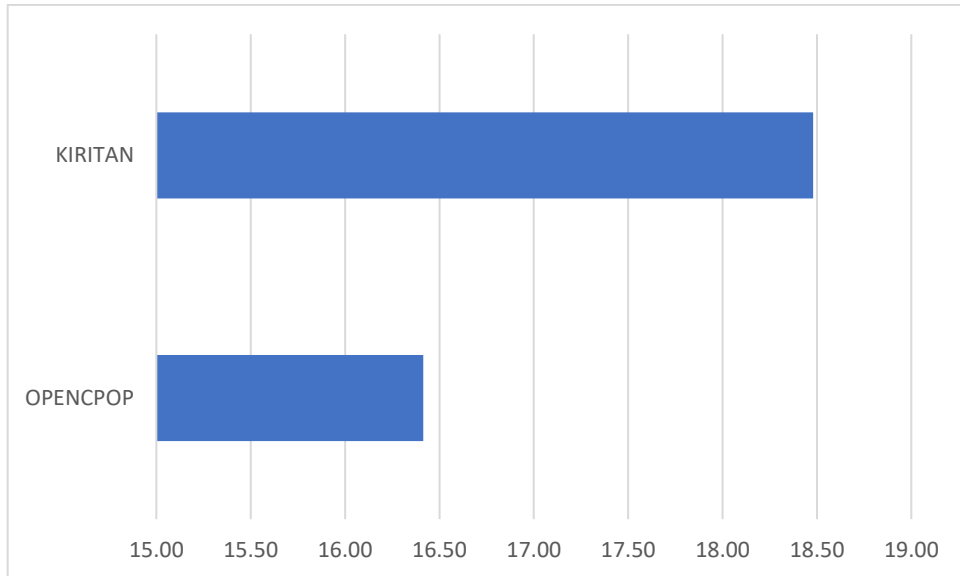


Figure 3.12: Baseline WER scores (Percent, Averaged)

Observation:

- While the system successfully transferred basic timbral characteristics, the output exhibited artifacts when processing Vietnamese audio. Due to the tonal complexity of the Vietnamese language (comprising six distinct tones), the baseline pipeline failed to extract accurate phonetic representations, resulting in muffled pronunciation and missing phonemes, which lead to high WER.

- An interesting anomaly was observed: in several instances, the synthesized audio is cleaner than the source audio, specifically for noisy source files, the converted outputs (kiritan and opencpop) exhibited improvements. This phenomenon can be attributed to the implicit denoising properties of the acoustic pipeline. The extraction of mel-spectrograms combined with the NSF-HiFiGAN waveform reconstruction effectively filters out background noise and microphone artifacts present in the original recordings. Furthermore, the diffusion model maps the source signal toward the high-fidelity acoustic distribution of the target speaker's studio-grade training corpus. Conversely, for source files that inherently possess high acoustic quality, the neural synthesis pipeline acts as a lossy compression mechanism, slightly degrading the micro-dynamics of the natural voice. This observation confirms that the framework not only converts timbre but also standardizes the overall signal quality based on the target training distribution.
- Nevertheless, this scores establish a baseline that'll be used to evaluates the final modified/ finetuned pipeline.

3.3.2.2. Phase 1: Shallow Diffusion Fine-Tuning

In this first optimization phase, the core Diffusion models was fine-tuned on the target speaker datasets. However, the system still utilized the pre-trained, unmodified ContentVec and NSF-HiFiGAN modules.

The Diffusion and the Naive prior model were fine-tuned from the provided pre-trained models to map linguistic representations to the specific target speaker's mel-spectrogram space. Both models were trained until the validation loss no longer improved.

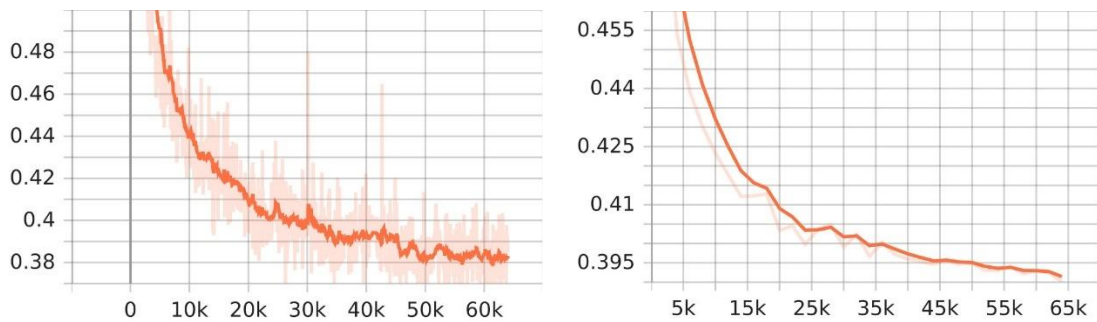


Figure 3.13: Training loss (left, 0.9 smoothing) and validation loss (right, 0.5 smoothing) of naive model. X axis is number of steps, Y axis is loss value.

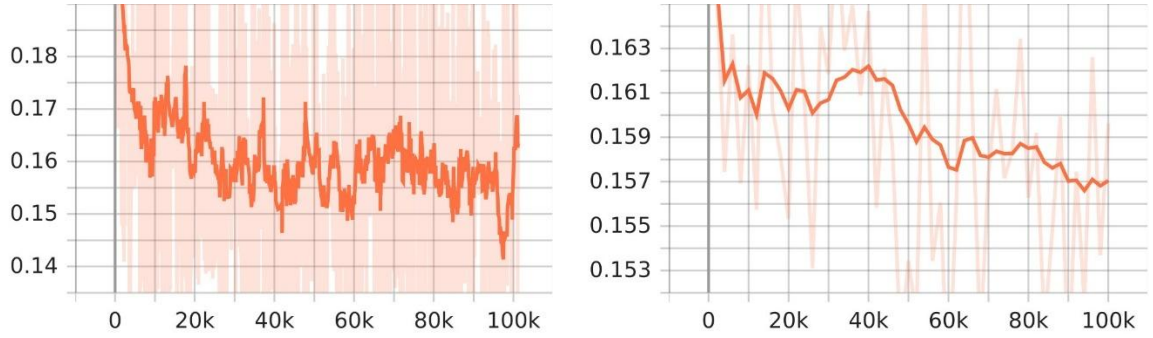


Figure 3.14: Training loss (left, 0.5 smoothing) and validation loss (right, 0.5 smoothing) of diffusion model. X axis is number of steps, Y axis is loss value.

Reference-based metrics will be used alongside with NISQA and speaker similarity metrics as the models now are finetuned to the desired target speakers.

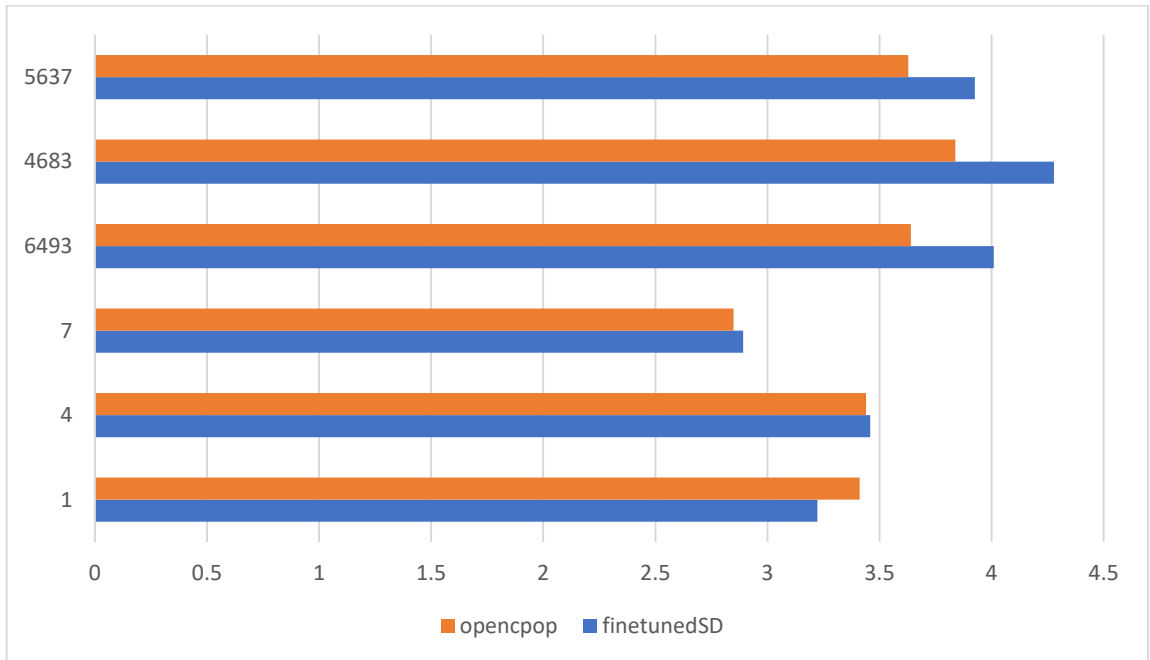


Figure 3.15: NISQA scores of the finetuned SD compared to the best baseline variant (6 samples)

Table 3.1: Reference-based metrics of the finetuned SD (Compared to the original audios) (6 samples)

Wav file/ Metrics	1	4	7	6493	4683	5637
Spectral Convergence	0.332	0.402	0.395	0.372	0.368	0.389
Log STFT Loss	2.614	1.768	2.131	0.925	1.340	1.423
MCD	190.710	247.922	258.116	176.511	219.321	234.674
F0 RMSE (Cents)	49.889	91.379	54.574	28.115	37.014	40.613
STOI	0.881	0.842	0.829	0.863	0.880	0.909
PESQ (Wideband)	1.882	1.781	1.475	1.965	1.847	1.894

Table 3.2: Speaker similarity scores of the phase 1 pipeline (Averaged of 100 samples)

	Female	Male
Mean similarity	0.752	0.747
Standard deviation	0.082	0.072
Min	0.496	0.551
Max	0.923	0.901

Observation:

- The domain adaptation of the diffusion denoiser successfully localized the timbral identity of the target speaker. NISQA metrics reveal significant perceptual improvements on specific audio samples. The speaker similarity scores also indicate that the models have optimized the speaker embeddings sufficiently (Achieving mean scores of 0.752 with female speaker and 0.747 with male speaker).
- These localized successes demonstrate that when the diffusion model receives optimal conditioning features, it possesses the generative capacity to reconstruct high-fidelity mel-spectrograms that closely mimic natural human timbre.
- On samples that achieved high NISQA scores, the strict reference-based evaluations yielded subpar results. The PESQ metric, which evaluates overall speech quality against the ground truth, hovered low across all samples, ranging from 1.47 to 1.89 (on a 4.5 scale). Correspondingly, the MCD remained high, between 190 and 258. This divergence indicates that while the audio might sound generally human-like, its specific spectral envelope deviates from the true target audio.
- The low STOI (ranging from 0.82 to 0.90) combined with the poor PESQ highlights a fundamental linguistic failure. Because the pre-trained ContentVec module is heavily biased toward English phonology, it failed to accurately map the complex tonal inflections (e.g., the “ngã” and “hỏi” tones) of the Vietnamese language. Consequently, the ContentVec fed flawed phonetic "pseudo-labels" to the diffusion network. The diffusion model successfully synthesized the target speaker's timbre, but the speaker was effectively forced to articulate distorted, unnatural syllables. This is also confirmed by listening to the inferred audio that have high notes, the model made robotic and/or flat tones because of the flawed encoder.

3.3.2.3. Phase 2: Encoder fine-tuning

Following the finetuning of the diffusion pipeline and the identification of the encoder as a potential source of error, the second phase focused entirely on adapting the phonetic extractor to address the persistent pronunciation errors observed in Phase 1.

Given the model's complex architecture and the need to preserve the generalization capability of the ContentVec model, this phase is divided into three experiments: full fine-tuning, partial fine-tuning (with all layers frozen except the last six), and an evaluation comparing 100-cluster and 500-cluster pseudo-labeling.

Configuration: For this evaluation, a controlled overfitting strategy is adopted to isolate the phonetic encoder. While all four modules are integrated into the pipeline, the Diffusion and Naive acoustic models are intentionally overfitted on a minimal subset of 20 samples (10 female and 10 male). By forcing these generative models to memorize a fixed acoustic distribution, the ContentVec encoder effectively becomes the sole independent variable in the pipeline, allowing us to evaluate its specific impact on pronunciation accuracy.

- **Sub-Experiments:**

1. **K-Means Pseudo-Label Resolution (100 vs. 500 Clusters):** Because ContentVec relies on discrete pseudo-labels for its self-supervised Masked Language Modeling objective, the granularity of the acoustic clustering directly dictates the model's phonetic vocabulary. An evaluation was conducted comparing a 100-cluster dictionary against a 500-cluster dictionary. Given the tonal density of the Vietnamese language (comprising six distinct tones that drastically alter semantic meaning), the 500-cluster dictionary (Which was used in the original Diffusion repo) was hypothesized to provide the necessary high-resolution targets for the network to differentiate subtle tonal shifts.

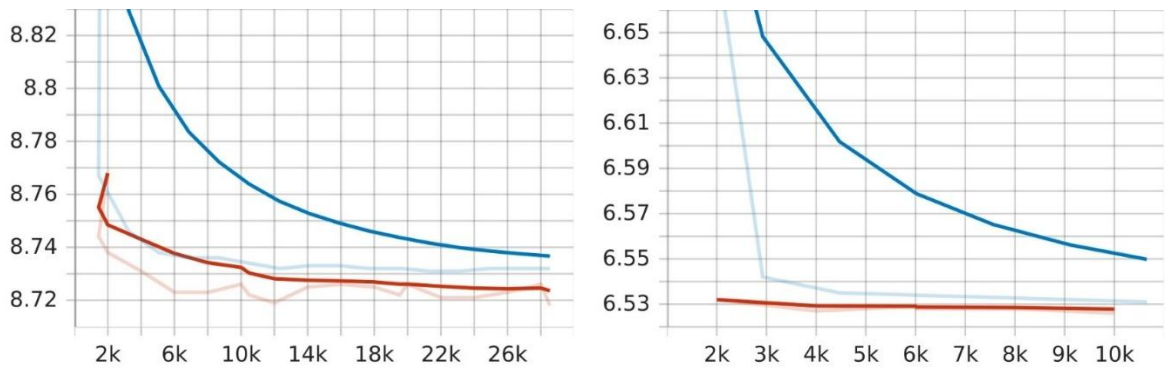


Figure 3.16: Training loss (blue) and validation loss (red) of 500 and 100 cluster model (0.8 smoothing). X axis is number of steps, Y axis is validation loss.

Although the validation loss of the 500 labels model is higher (8.72 compares to 6.5) than the 100 labels, but this does not necessary make the 500 labels model worse, as seen in the results between two models:

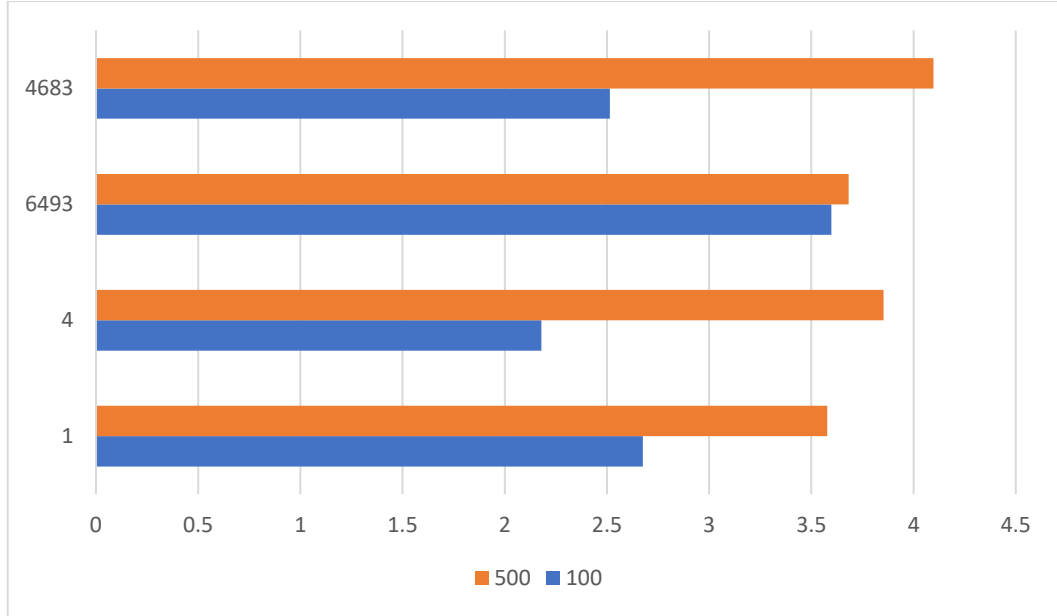


Figure 3.17: NISQA scores between 100 and 500 clusters labeling

The differences between 2 models is clear, the 100-cluster configuration exhibited severe tonal ambiguity. Due to the high tonal density of the Vietnamese language (comprising six distinct pitch inflections), the low-resolution clustering mathematically forced phonetically distinct but acoustically similar syllables (e.g., words differing only by a rising or falling tone like “*á*” and “*à*”) into identical cluster IDs. Consequently, the output audio suffered from flattened prosody and incorrect tonal articulation. Conversely, the 500-cluster configuration provided a sufficiently expansive vocabulary, enabling the encoder to capture subtle phonetic boundaries. With the NISQA score massive different, the reference-based metrics will be skipped as it is not needed.

- 2. Full vs. Partial Fine-Tuning (Layer Freezing Policy):** To determine the optimal balance between preserving generalized acoustic knowledge and learning language-specific phonetics, two weight-updating strategies were compared.

The full fine-tuning model used here is the 500-cluster model from the previous point.

For the partial model, because the original implementation of Contentvec provided in the official repository trains the entire model architecture by default, therefore in order to optimize/ fine-tune the model for Vietnamese phonetics while preventing catastrophic forgetting of the pre-trained acoustic representations, a partial freezing strategy was proposed. Specifically, the objective is to freeze the feature extractor with *model.feature_grad_mult=0.0* and all transformer layers except the final six layers with custom logics.

Since the base repository does not natively support freezing specific encoder layers, modifications were made to the core model configuration. A new parameter,

freeze_encoder_layers, and additional logic was integrated into the ContentvecConfig dataclass to control the number of layers that bypass the gradient update during the fine-tuning phase:

```
freeze_encoder_layers: int = field(
    default=0, metadata={"help": "number of transformer layers
to freeze"})

...

if cfg.freeze_encoder_layers > 0:
    logger.info(f"Freezing the first
{cfg.freeze_encoder_layers} transformer layers.")
    for i in range(cfg.freeze_encoder_layers):
        for param in self.encoder.layers[i].parameters():
            param.requires_grad = False
```

Figure 3.18: Additional parameter and logic for freezing the transformer layers

All the layers, CNN included, except for the last 6 layers of the Transformer block are frozen for this experiment. The reason here is the provided pre-trained model was trained with the config of 3-3-6, 3 first layers had speaker embeddings, 3 layers after doesn't, and the output of the 6-th layer was where the contrastive loss is applied, which makes the last 6 layers are where the phonetic contents are learned (See Figure 2.2).[11]

Both models were trained until plateau.

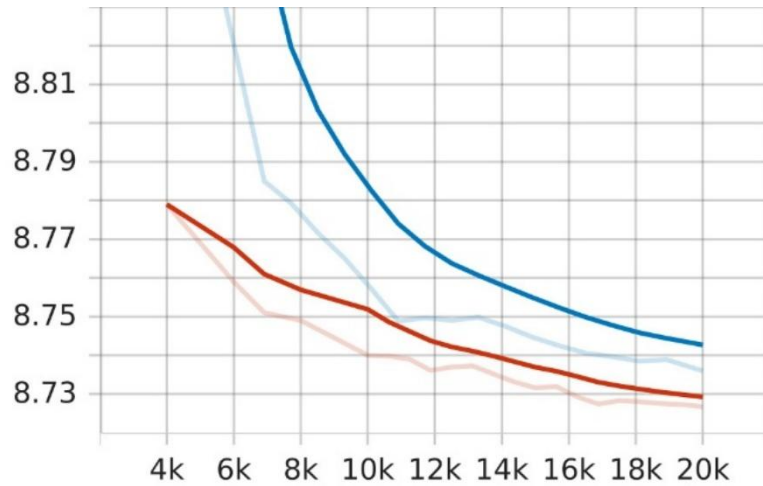


Figure 3.19: Training loss (blue) and validation loss (red) of Partial Fine-Tuning model (0.8 smoothing). X axis is number of steps, Y axis is validation loss.

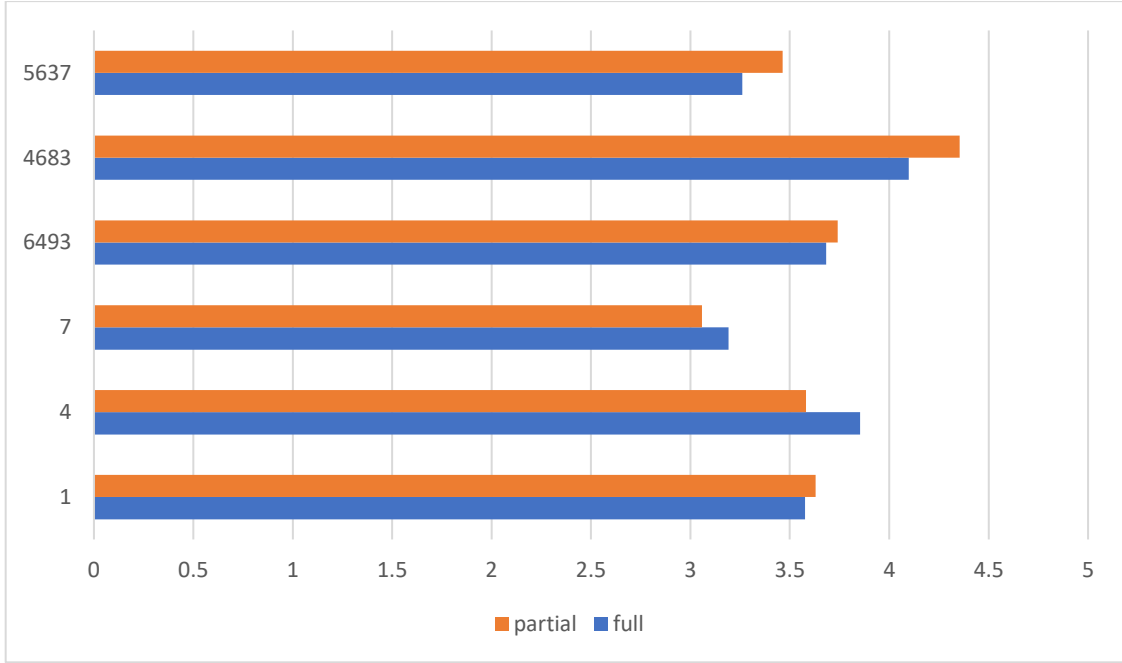


Figure 3.20: NISQA scores between partial and full fine-tuning

**Table 3.3: Reference-based metrics between partial and full fine-tuning
(Compared to the original audios, average of 6 samples)**

	Full Fine-Tuning	Partial Fine-Tuning
Spectral Convergence	0.376	0.389
Log STFT Loss	1.703	1.698
MCD	190.189	197.448
F0 RMSE (Cents)	44.063	77.67
STOI	0.876	0.869
PESQ (Wideband)	1.841	1.769

The validation loss of both models were within the margin of error.

While objective metrics favor the full model, the neural-based MOS from NISQA indicates a preference for the partial configuration in the majority of samples (e.g., 6493, 5637, and 4683).

Observations during inference suggest that the partial model produces a more vibrant and natural timbre. In contrast, the full model exhibits signs of "spectral flattening", a phenomenon where the voice sounds overly compressed or "flat", especially at high notes, which was also observed in Phase 1 using the pre-trained ContentVec. By freezing the initial layers of ContentVec, the model can retain the rich, high-level acoustic representations from the large-scale pre-training phase,

but still suffers from the “flat” tone error in some cases, often associated with spectrogram collapse and formant merging:

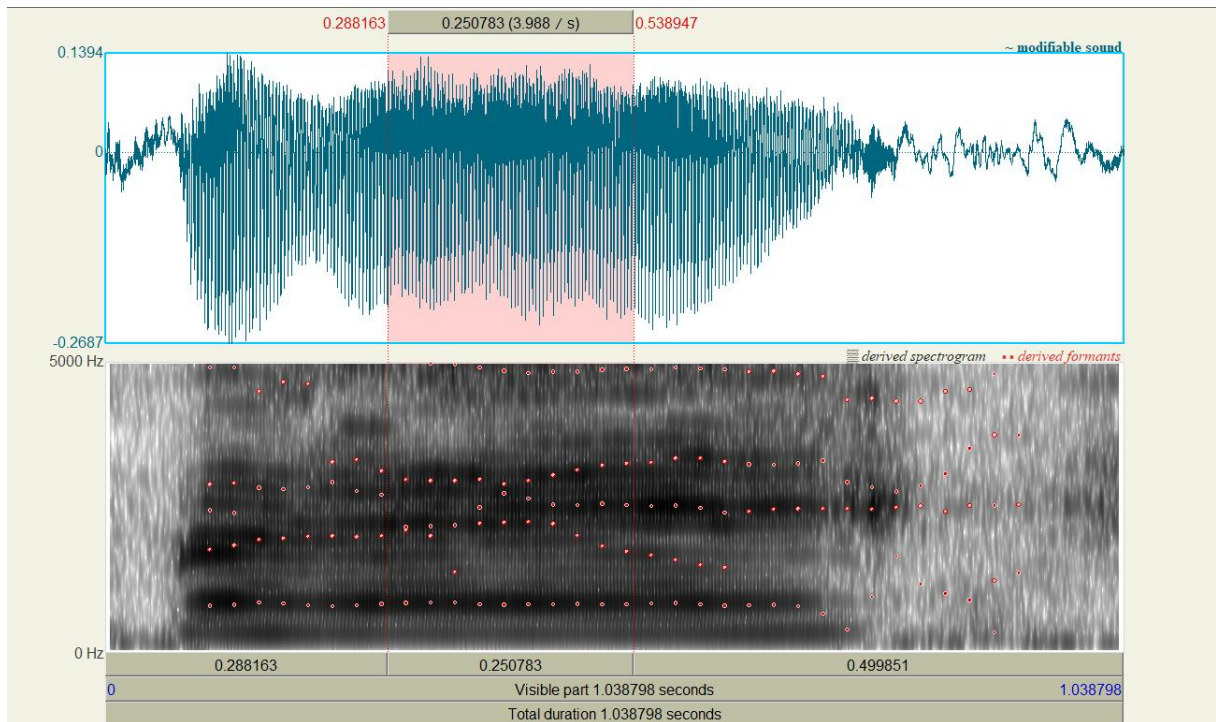


Figure 3.21: An example of spectral smearing, full model.

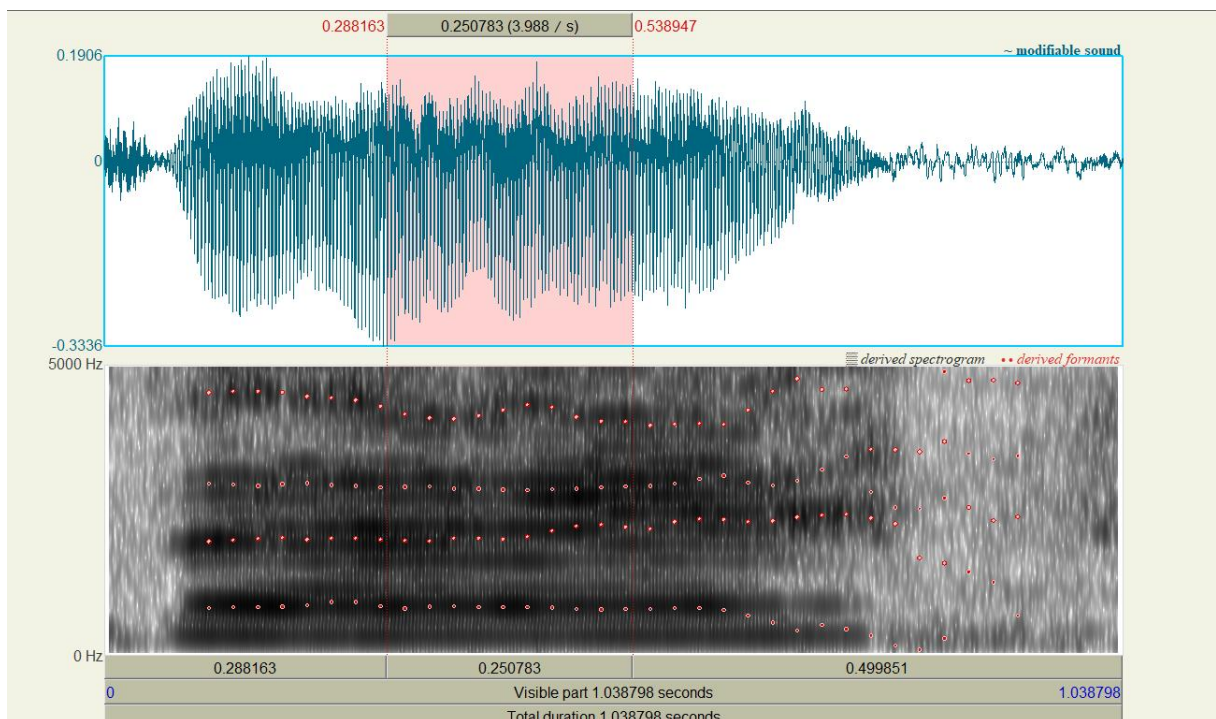


Figure 3.22: The same vowel from partial model.

In this example, both models were inferring the vowel “è” as in “vè”. The spectrogram of the full model collapsed/smeared the middle bands where F2 and

F3 (the second and third red dots/black lines from bottom to top) reside, while also pushing F1 (the bottom red dots/black line) upward, causing the vowel to be pronounced as “è.” In contrast, the partial model kept the F1 and F2 bands clearly separated, resulting in the correct pronunciation of “ê”. The reason for this is hypothesized to be that the training targets (labels) for ContentVec are generated via K-means clustering using the original pretrained model. These labels inherently carry the "phonetic bias" of the source language (e.g., English or Chinese) the model is pre-trained on. The base Hubert models also suffers from this issue, which needs extensive iterative training to reduce it.

The lower MCD in the full model suggests a tighter mathematical fit to the target spectral envelope. However, this "tight fit" often leads to the loss of fine harmonic details, resulting in the aforementioned flattened sound. The slightly higher MCD in the partial model actually reflects a more complex and dynamic spectral structure that sounds more pleasing to the human ear.

While the full model shows superior stability in F0 RMSE, the partial model provides a better trade-off and will be chosen as the encoder moving forward. The minor loss in pitch precision is outweighed by the significant gain in vocal richness and the “mostly” elimination of "flat" artificial distortions.

3. Early Stopping

While better, the partial fine-tuned still makes “flat” sounds at high notes, this is predicted to be due to excessive training:

- Loss Paradox: If training is continued until the validation loss reaches its minimum, the model will perfectly recreate the original model's errors. It will force Vietnamese vowels back into the incorrect "bins" defined by the pretrained clusters.
- Catastrophic Forgetting: Over-optimization leads to the model "forgetting" the high-frequency nuances of the original voice, resulting in strident, metallic, or "flattened" audio quality.

However, standard metrics may not detect the subtle shift from phonetic improvement to acoustic degradation. Therefore, the determination of the optimal checkpoint will also be conducted through Cross-Checkpoint Inference method, evaluated by subjective listening tests.

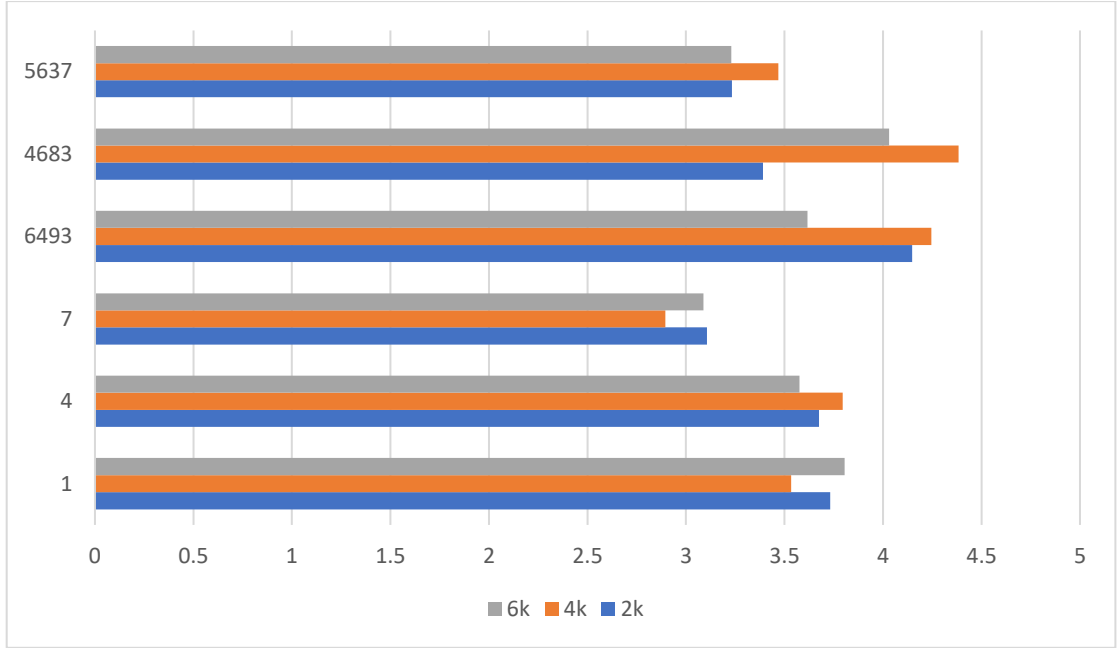


Figure 3.23: NISQA scores between checkpoints (2k-4k-6k steps)

Table 3.4: Reference-based metrics between checkpoints (Compared to the original audios, average of 6 samples)

	2k steps	4k steps	6k steps
Spectral Convergence	0.359	0.356	0.362
Log STFT Loss	1.393	1.685	1.395
MCD	194.828	192.961	196.895
F0 RMSE (Cents)	38.199	44.822	65.317
STOI	0.874	0.883	0.87
PESQ (Wideband)	1.756	1.852	1.774

The overall better scores of the 4k steps checkpoint, combined with better pronunciation in subjective listening shows that it is the sweet spot in fine-tuning the encoder, and will be used as the final encoder in current pipeline.

3.3.2.4. Phase 3: Vocoder/Decoder Fine-tuning

The vocoder generally doesn't need to be focused as much as the encoder as modern universal vocoders are robust enough to reconstruct high-quality audio from diffusion-generated spectrograms without the intensive computational overhead of per-subject finetuning. The results between the pre-trained and fine-tuned models further confirm this claim.

The chosen model is NSF-HiFiGan, fine-tuned until no improvement is observed. The same overfitting strategy is implemented with previously selected 4k steps encoder.

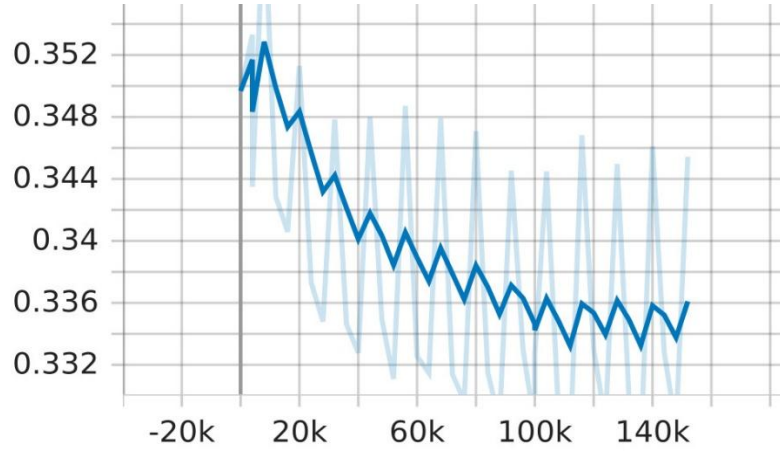


Figure 3.24: Total Validation loss of fine-tuned model (0.8 smoothing). X axis is number of steps, Y axis is validation loss.

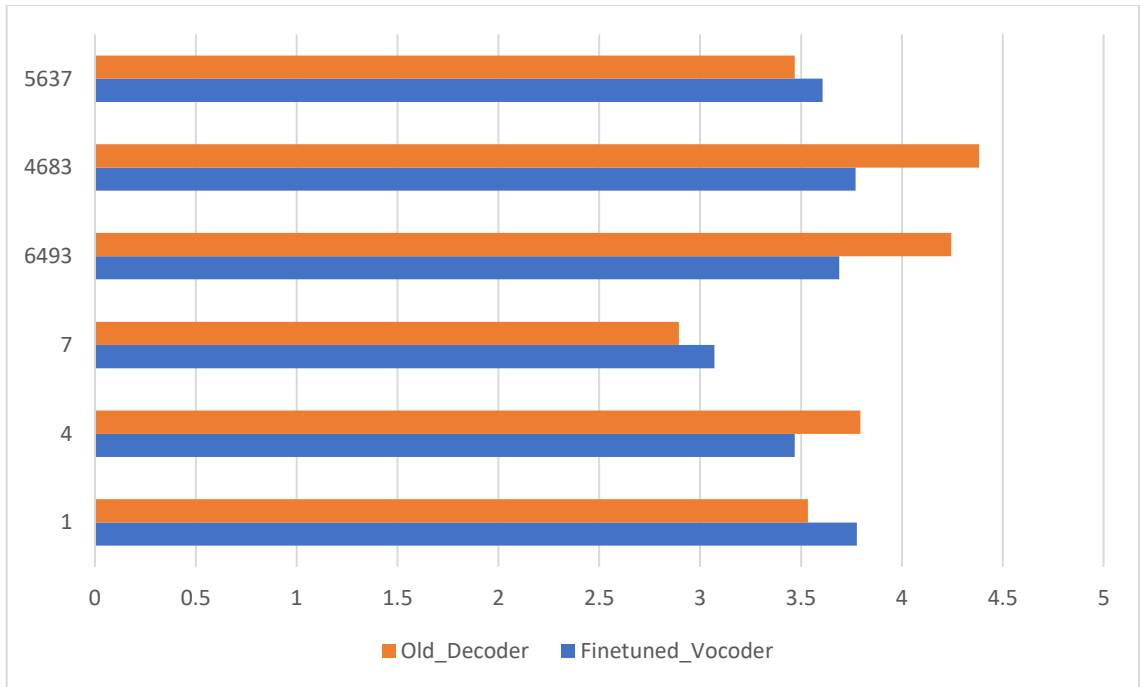


Figure 3.25: NISQA scores between fine-tuned and stock NSF-HiFiGan

Table 3.5: Reference-based metrics between fine-tuned and stock NSF-HiFiGan (Compared to the original audios, average of 6 samples)

	Fine-tuned	Stock (Pre-trained)
Spectral Convergence	0.348	0.351
Log STFT Loss	1.378	1.685
MCD	188.056	192.999
F0 RMSE (Cents)	47.242	52.991
STOI	0.873	0.886
PESQ (Wideband)	1.878	1.842

Again, a contrast in performance scores (Reference-based vs NISQA) is observed. As explained in Contentvec section, this difference won't contribute too much to the choice of model. However, unlike the Contentvec's situation, no major difference is observed in subjective listening tests, mostly falling within margin of error. Therefore, the final model mostly comes down to preference, either better naturalness of the stock (pre-trained) NSF-HiFiGan model, or better imitation of the target voices of the fine-tuned model.

3.3.3. Final results

After finetunings, evaluations and necessary modifications, the final pipeline can be put together and against the baseline. The final pipeline consists of the 4k steps Contentvec model, pre-trained stock NSF-HiFiGan, and shallow diffusion combo models finetuned on the data preprocessed with the new Contentvec. Both Diffusion and Naive models were trained until plateau. This experiment used the same 100 audio samples as the baseline described in Section 3.3.2.1.

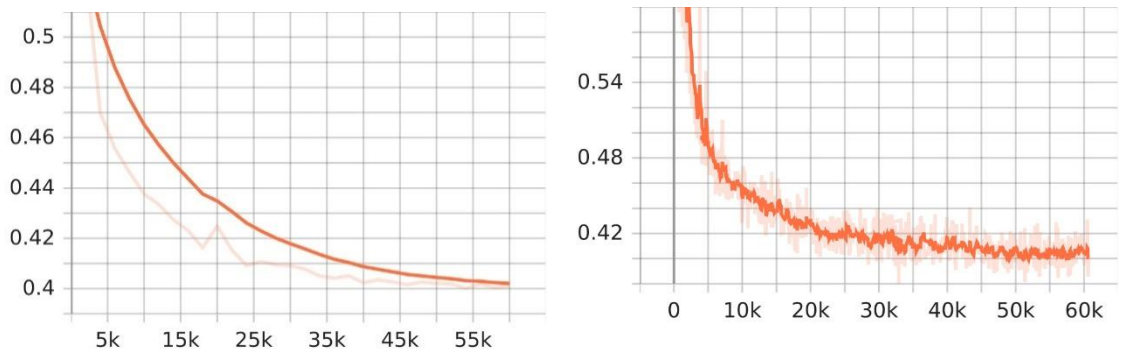


Figure 3.26: Training loss (left) and validation loss (right) of naive model. X axis is number of steps, Y axis is loss value, 0.8 smoothing.

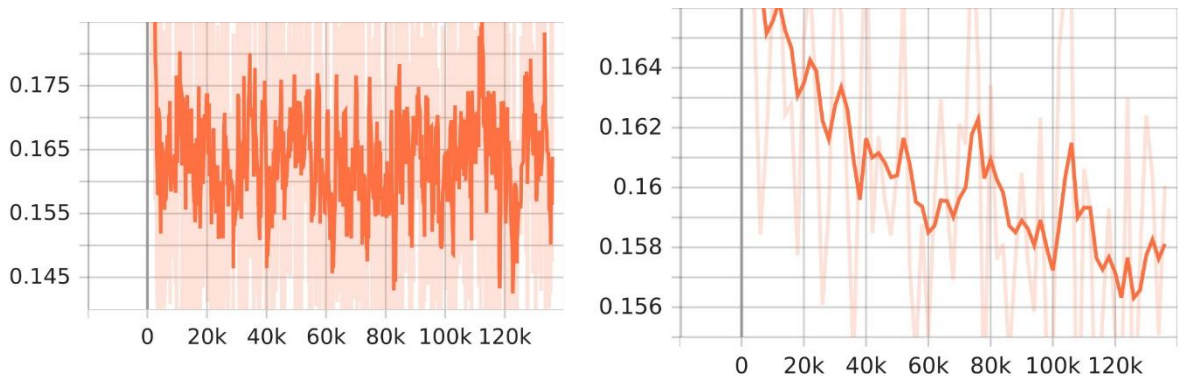


Figure 3.27: Training loss (left) and validation loss (right) of diffusion model. X axis is number of steps, Y axis is loss value, 0.8 smoothing.

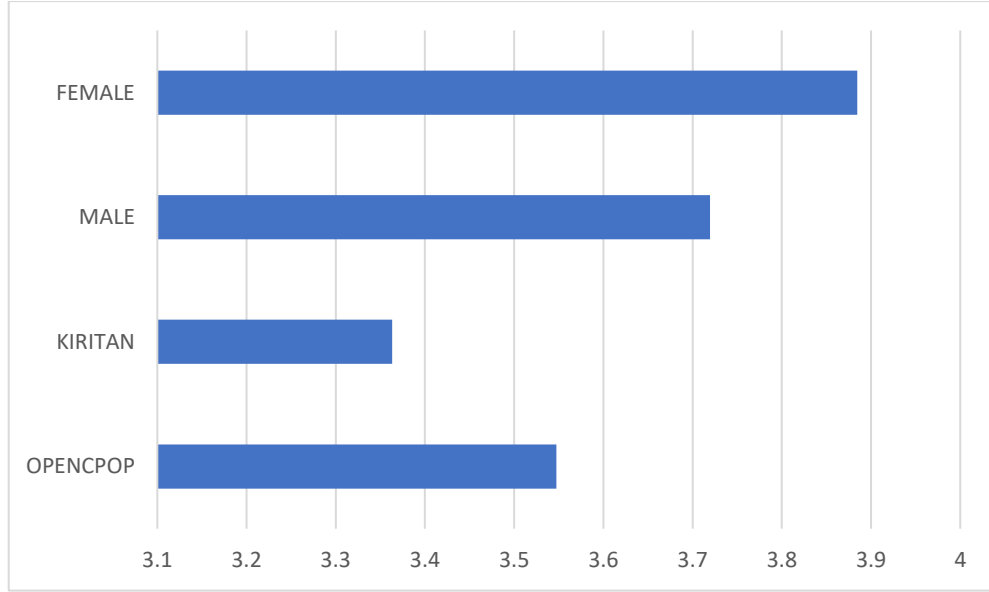


Figure 3.28: NISQA scores between final pipeline and baseline (averaged)

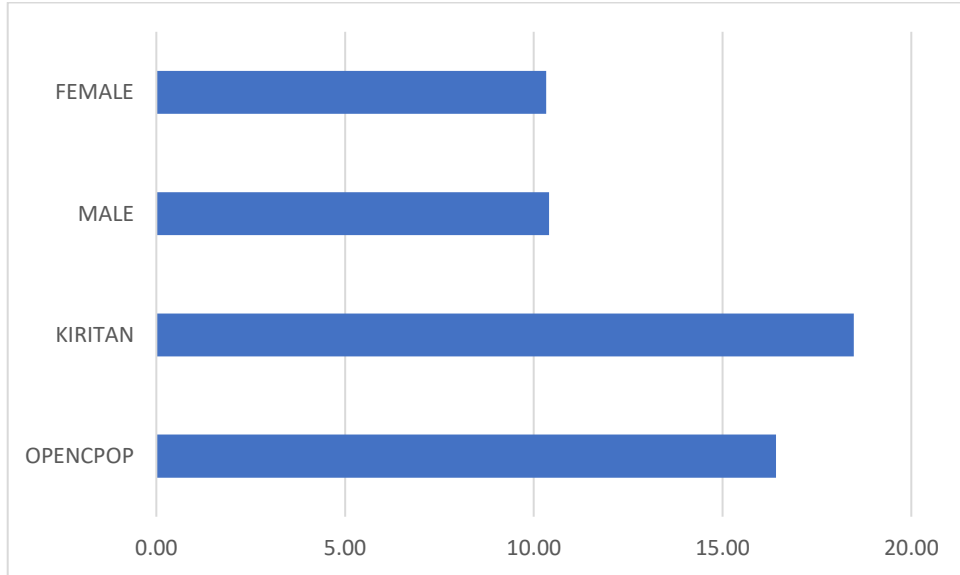


Figure 3.29: WER scores between final pipeline and baseline (averaged)

The proposed models outperformed the baseline across all evaluation metrics, demonstrating substantial improvements in overall synthesis quality. Despite utilizing a non-optimal ContentVec encoder model (Stated in “Early stopping” (section 3.3.2.3.2) and it’s scores in Table 3.2)-which introduced minor auditory artifacts into the inferred audio-the final models still delivered structurally more coherent speech than the baseline. Consequently, this led to a notable enhancement in performance for Vietnamese speech processing. Specifically, speech naturalness scores increased from a baseline range of 3.36-3.54 to 3.71-3.88 (an average increase of approximately 0.35), while the WER dropped significantly from 18.48%-16.42% down to 10.40%-10.33% (an average absolute decrease of roughly 7.08%).

The simultaneous improvement in both speech naturalness and content preservation highlights the robustness of the implemented framework. In Voice Conversion tasks, there is often an inherent trade-off between speaker similarity, acoustic quality, and phonetic intelligibility. The substantial reduction in WER indicates that the model successfully preserved the underlying linguistic structure of the source speech. Since Vietnamese is a tonal language where minor acoustic distortions can alter the semantic meaning of a word, a WER of approximately 10.3% proves that the phonetic units extracted by the framework are highly stable and clear enough for a state-of-the-art ASR model (OpenAI Whisper) to decode accurately.

However, the presence of the aforementioned noise/artifact points to a clear bottleneck in the current pipeline. While the current results validate the effectiveness of the architecture, further optimization of the ContentVec encoder-such as self-supervised pre-training or fine-tuning iteratively on a larger, high-quality Vietnamese speech corpus-could drastically minimize these artifacts. Resolving this encoder-level limitation is expected to close the remaining gap in speech naturalness and elevate the system closer to ground-truth acoustic performance.

Table 3.6: Reference-based metrics between final pipeline and stock pipeline(Phase 1). (Compared to the original audios, average of 10 samples)

	Final	Stock
Spectral Convergence	0.415	0.380
Log STFT Loss	1.711	1.706
MCD	228.168	228.273
F0 RMSE (Cents)	41.961	88.821
STOI	0.868	0.866
PESQ (Wideband)	1.788	1.744

Table 3.7: Speaker similarity scores of the final pipeline (Averaged of 100 samples)

	Female	Male
Mean similarity	0.751	0.753
Standard deviation	0.081	0.071
Min	0.547	0.552
Max	0.891	0.894

The final pipeline has learned to control pitch better, resulting in lower F0 RMSE, more natural intonation and reducing the "robotic" effect or tone errors in Vietnamese and other metrics stay within margin of errors compare to the stock pipeline except for Spectral Convergence score. The speaker similarity scores are also average at an acceptable and virtually the same scores as phase 1 (0.751 for female and 0.753 for male), indicating the diffusion models don't suffer from the encoder situation. Although, The use of non-optimal encoder resulting in presence of spectral noise or also known as "grainy"/ "dirty" audio, combining with sub-optimal datasets, higher Spectral Convergence score, as well as lower speaker similarity is anticipated and inevitable. Though overall, the final pipeline still shows improvements in inferred audio compare to the baseline despite it's flaws.

3.4. Other personal modifications and clarifications

This section covers the remaining modifications made to the pipeline or clarifies mentioned techniques that were not addressed in the sections above.

All custom scripts mentioned in this project are available in the author's repositories, which are forked from and modified versions of the original repositories, unless stated otherwise.

3.4.1. Contentvec

Beside the custom script (`create_contentvec_dict.py`) mentioned in the preprocessing section, another custom script (`convertToLegacy.py`) is written and added to the repository in order to convert the fine-tuned checkpoints to the usable model format that the Diffusion repository requires. What this script does is cut off the layers' weights as well as neccessary parameters from the fine-tuned model that are not needed and can cause incompatibility with the diffusion pipeline's requirements.

3.4.2. Diffusion

In order to do inference from male to targeting female voice and vice-versa automatically instead of manually, a custom script (`clamp.py`) is written to shift the input audio's F0 contour by a number of semitones calculated on the target voice's F0 quantiles which are calculated in custom script `getf0Quantile.py` on the already extracted F0 npy files from the preprocessing step. However, the shifted semitone can also be inputted manually if needed. Another function which compresses all the f0 values that are outside of the model's extrapolate range is also implemented inside the `clamp.py` script. These addons are all injected into the base `extract_f0` code and described as follow:

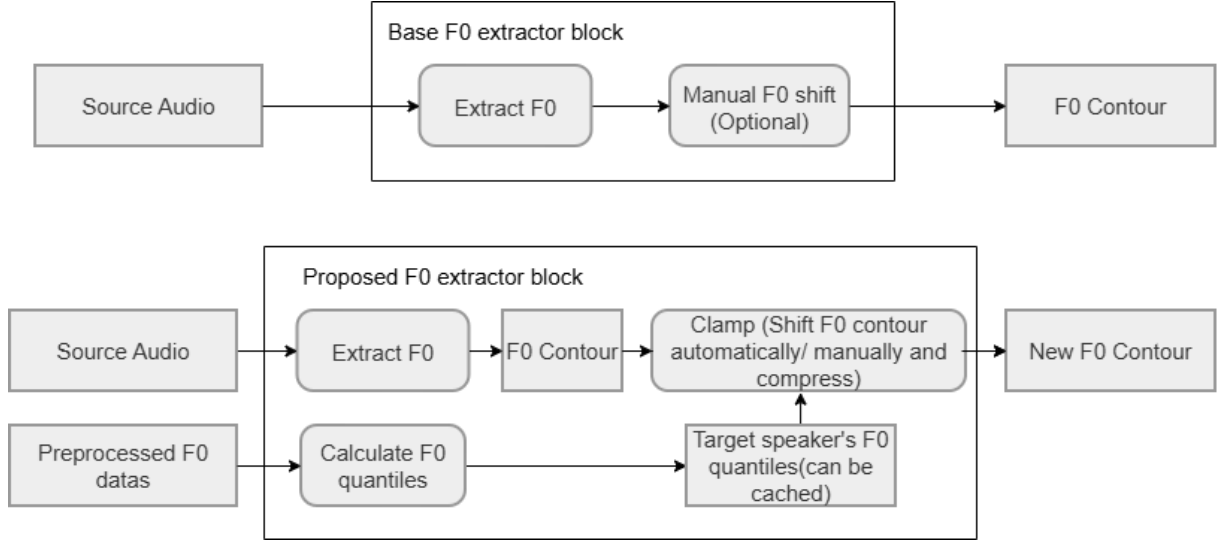


Figure 3.30: Proposed F0 extractor block

- **Speaker-Specific Statistical Analysis:**

Before inference, the target speaker's vocal range is statistically modeled using their training dataset. For a speaker S , we define the set of voiced frames as $F_S = \{f \in Dataset_S | f > 65 \text{ Hz}\}$. A quantile-based approach is used to define robust boundaries, effectively ignoring tracking outliers:

$$Hz_{min} = Q(F_S, 0.05)$$

$$Hz_{max} = Q(F_S, 0.95)$$

Where $Q(X, p)$ is the p -th quantile of the distribution X . These values are cached in a global dictionary Q_{cache} to optimize inference latency for multi-file processing. Exported model can integrate this values, or parse them to external config files for use when original training datasets are not available.

- **Median-Based Pitch Contour Shifting:**

To align the source pitch with the target speaker's range, a global semitone shift Δs is calculated. Unlike mean-based shifting, using the median frequency f_{med} provides robustness against octave jumps and transient errors.

The raw semitone shift is calculated based on the boundary violation of the median:

$$\Delta s_{raw} = \begin{cases} 12 \cdot \log_2\left(\frac{Hz_{min}}{f_{low}}\right) & \text{if } f_{med} < Hz_{min} \\ 12 \cdot \log_2\left(\frac{Hz_{max}}{f_{high}}\right) & \text{if } f_{med} > Hz_{max} \\ 0 & \text{otherwise} \end{cases}$$

To maintain the structural integrity of the model's extrapolation limits, the final shift is clipped:

$$\Delta s = \text{clamp}(\Delta s_{raw}, -s_{max}, s_{max})$$

The adjusted contour is then computed as:

$$f_{shifted} = f_{original} \cdot 2^{\frac{\Delta s}{12}}$$

- Non-Linear Logarithmic Compression:

To handle residual extreme frequencies that may still exist after shifting (such as very high falsettos or deep fry), a non-linear compression in the logarithmic domain is applied. This prevents "robotic" artifacts by softly "squashing" the F0 peaks rather than hard-clipping them.

Let $L = \ln(f + \epsilon)$ be the log-frequency. The compressed log-frequency L' is defined by the piecewise function:

$$L' = \begin{cases} L_{min} + \gamma_{low}(L - L_{min}) & \text{if } L < L_{min} \\ L_{max} + \gamma_{high}(L - L_{max}) & \text{if } L > L_{max} \\ L & \text{otherwise} \end{cases}$$

Where γ_{low} and γ_{high} are compression ratios (set to 0.25 and 0.5 respectively). The final F0 is reconstructed via $f_{final} = \exp(L')$.

3.4.3. Evaluation methods

- NISQA scores are calculated with the cloned official repository.
- The reference-based metrics are implemented through librosa [23], pystoi [24] and PESQ [25] libraries in custom python script `stft_loss_check.py`.
- WER scores are calculated with jiwer[26] python library.

3.5. Chapter summary

This chapter detailed the experimental methodologies and the comprehensive evaluations used to compare the baseline and fine-tuned models. The analysis presented the dataset preparation, implementation configurations, and the specific training procedures applied to adapt the pre-trained systems for Vietnamese speech conversion. By benchmarking the baseline configurations against the fine-tuned iterations, the evaluation demonstrated the role of domain-specific training in enhancing overall synthesis quality.

The chapter also examined the performance differences through assessments utilizing both objective metrics, such as MCD and F0 error measurements, alongside subjective MOS testing. These results highlighted how effectively the fine-tuned models capture the linguistic complexity, phonetic nuances, and distinct tonal characteristics of the Vietnamese language compared to the original baselines. It also highlights that the modified encoder finetuning process plays a critical role in this pipeline.

CONCLUSIONS AND RECOMMENDATIONS

I. Contributions

- Modified and extended the original ContentVec and Shallow Diffusion implementations to improve compatibility with Vietnamese speech and with limited experiments on singing voice data.
- Developed and documented a complete and reproducible ContentVec training and fine-tuning pipeline, addressing missing training components and limited training guidance in the original repository.
- Conducted one of the first fine-tuning and evaluation studies of ContentVec on Vietnamese speech data, demonstrating its effectiveness for Vietnamese speech representation learning.
- Fine-tuned and optimized Shallow Diffusion, and NSF-HiFiGAN models on Vietnamese datasets for voice conversion tasks.
- Evaluated the performance of the original voice conversion pipeline with fine-tuned components using both reference-based, non-reference-based metrics and audio demonstrations .
- Provided a reproducible implementation and experimental framework that can serve as a baseline for future Vietnamese voice conversion research.

II. Limitations

1. Optimization Constraints of the ContentVec Encoder

The performance of the feature extractor, ContentVec, was found to be bottlenecked by its reliance on pre-trained representations. Since the original K-means clustering and pre-training stages were conducted on massive English or Chinese datasets, the resulting acoustic centroids do not fully capture the unique tonal nuances and glottal characteristics of Vietnamese. Consequently, the encoder's ability to extract speaker-independent features is limited by this cross-lingual dependency, preventing it from reaching its theoretical maximum potential for native Vietnamese tasks.

2. Data Scarcity and Quality Issues

The quality of the final synthesis was significantly influenced by the limitations of the available Vietnamese speech corpora. The project faced several data-related challenges:

- Lack of Diversity: Current public Vietnamese datasets lack the phonetic variety and emotional range required for robust model generalization.
- Noise Interference: Due to the scarcity of studio-quality "clean" datasets, the inclusion of samples with minor background noise or recording artifacts was

inevitable. This occasionally resulted in the model "learning" these impurities, leading to subtle noise artifacts in the converted output.

- **Resource Constraints:** The computational overhead required for exhaustive models pre-training was restricted by the available hardware, limiting the depth of exploration for the Diffusion model as well as the pipeline.

3. The Necessity of the Training Trade-off

A decision was made to stop the training of the ContentVec-based pipeline before reaching the absolute minimum loss. This early stopping and partial layer freezing approach served as a vital trade-off to prevent the model from overfitting to the limited and the minor imperfections caused by less than ideal clustering of dataset. By stopping early as well as freezing all but the last 6 transformers layers, the model preserved the generalized acoustic knowledge from the pre-training phase, effectively mitigating "spectral flattening" and ensuring a more natural, albeit not perfect vocal output.

III. Recommendations

Based on the findings and limitations of this project, the following recommendations are proposed to enhance the effectiveness of Vietnamese Voice Conversion systems:

- **Data Standardization:** It is highly recommended that academic institutions and the AI community in Vietnam collaborate to build a standardized, open-source, and high-quality "Clean Speech" dataset. This would provide a foundation for training encoders that are more sensitive to Vietnamese tonality.
- **Localized Pre-training:** Future implementations should prioritize pre-training the ContentVec or similar encoders directly on Vietnamese phonetics rather than relying on cross-lingual transfer from English or Chinese models to fully unlock the potential of the Diffusion-SVC, as well as other Voice Conversion pipelines. Alternatively, iterative training is also useful if the mentioned solution is not viable.
- **Hybrid Training Strategies:** For developers working with limited resources, a hybrid approach for Contentvec-combining partial layer freezing with early stopping-is recommended to maintain perceptual naturalness and avoid the "flattening" effect in low-resource language adaptation, unless the previous points is satisfied.

REFERENCES

- [1] Luong Thi Hieu, Vu Hai Quan (2016), “A non-expert Kaldi recipe for Vietnamese Speech Recognition System,” *Proceedings of the Third International Workshop on Worldwide Language Service Infrastructure and Second Workshop on Open Infrastructures and Analysis Frameworks for Human Language Technologies (WLSI/OIAF4HLT2016)*, Osaka, Japan, pp. 51-55.
- [2] Tran Duc Chung (2020). “FPT Open Speech Dataset (FOSD) - Vietnamese,” *Mendeley Data*, V4.
- [3] Pham Nguyen Ngoc Bao, “VieNeu-TTS: Vietnamese Multi-Speaker Text-to-Speech Dataset,” Hugging Face, 2025, <<https://huggingface.co/datasets/pnnbao-ump/VieNeu-TTS>> Accessed May 30, 2026
- [4] doof-ferb, “infore1_25hours: Vietnamese Speech Dataset,” Hugging Face, 2023. <https://huggingface.co/datasets/doof-ferb/infore1_25hours> Accessed May 30, 2026
- [5] Nguyen Van Dinh, Thanh Chi Dang, Luan Thanh Nguyen, et al. (2024), “Multi-Dialect Vietnamese: Task, Dataset, Baseline Models and Challenges,” *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Miami, Florida, USA, pp. 7476-7498.
- [6] Nguyen Thi Thu Trang, Huu Tuong Tu, Le Hoang Anh Tuan (2025), “The 2025 VLSP Task on Vietnamese Voice Conversion: Overview and Preliminary Results,” *Proceedings of the 11th International Workshop on Vietnamese Language and Speech Processing*, Hanoi, Vietnam, pp. 90-98.
- [7] Sisman B., Yamagishi J., King S., et al. (2021), “An overview of voice conversion and its challenges: From statistical modeling to deep learning,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29, pp. 132-157.
- [8] Ho J., Jain A., Abbeel P. (2020). Denoising Diffusion Probabilistic Models. *Advances in Neural Information Processing Systems (NeurIPS 2020)*.
- [9] Liu J., Li C., Ren Y., et al. (2022), “DiffSinger: Singing Voice Synthesis via Shallow Diffusion Mechanism,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(10), pp. 11020-11028.
- [10] Phạm Ngọc Phương (2023), “Nghiên cứu phát triển hệ thống thích nghi giọng nói trong tổng hợp tiếng Việt và ứng dụng,” Luận án Tiến sĩ, Học viện Khoa học và Công nghệ.
- [11] Qian K., Zhang Y., Gao H., et al. (2022), “ContentVec: An Improved Self-Supervised Speech Representation by Disentangling Speakers,” *Proceedings of the 39th International Conference on Machine Learning (ICML 2022)*, pp. 18003-18017.
- [12] Wang X., Yamagishi J. (2019), “Neural Harmonic-plus-Noise Waveform Model with Trainable Maximum Voice Frequency for Text-to-Speech Synthesis,” *Proceedings of the 10th ISCA Speech Synthesis Workshop (SSW 10)*, pp. 1-6.

- [13] Kong J., Kim J., Bae J. (2020), “HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis,” *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS 2020)*.
- [14] CNChTu, “Diffusion-SVC,” GitHub repository.
<<https://github.com/CNChTu/Diffusion-SVC>> Accessed May 30, 2026
- [15] openvpi, “SingingVocoders,” GitHub repository.
<<https://github.com/openvpi/SingingVocoders>> Accessed May 30, 2026
- [16] Arik S.O., Jun H., Diamos G. (2019), “Fast Spectrogram Inversion Using Multi-Head Convolutional Neural Networks,” *IEEE Signal Processing Letters*, 26(1), pp. 94-98.
- [17] Kubichek R. (1993), “Mel-Cepstral Distance Measure for Objective Speech Quality Assessment,” *Proceedings of the IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM 1993)*, pp. 125-128.
- [18] Rix A.W., Beerends J.G., Hollier M.P., et al. (2001). “Perceptual Evaluation of Speech Quality (PESQ): A New Method for Speech Quality Assessment of Telephone Networks and Codecs,” *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2001)*, pp. 749-752.
- [19] Taal C.H., Hendriks R.C., Heusdens R., et al. (2010), “A Short-Time Objective Intelligibility Measure for Time-Frequency Weighted Noisy Speech,” *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2010)*, pp. 4214-4217.
- [20] Mittag G., Naderi B., Chehadi A., et al. (2021), “NISQA: A Deep CNN-Self-Attention Model for Multidimensional Speech Quality Prediction with Crowdsourced Datasets,” *Proceedings of Interspeech 2021*, pp. 2127-2131.
- [21] Ravanelli M., Zhong J., Pascual S., et al. (2021), “SpeechBrain: A General-Purpose Speech Toolkit,” *arXiv preprint arXiv:2106.04624*.
- [22] Nguyen Vinh Huy, Nguyen Dinh Thuan, “Dolly-Audio: Vietnamese Multi-Speaker High-Quality Speech Corpus,” Dolly AI Team, 2025.
<<https://huggingface.co/datasets/dolly-vn/dolly-audio-1000h-vietnamese>>
Accessed May 30, 2026
- [23] McFee B., Raffel C., Liang D., et al. (2015), “librosa: Audio and Music Signal Analysis in Python,” *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, pp. 18–24.
- [24] mpariente, “pystoi,” GitHub repository. <<https://github.com/mpariente/pystoi>>
Accessed May 30, 2026
- [25] ludlows, “PESQ,” GitHub repository. <<https://github.com/ludlows/PESQ>>
Accessed May 30, 2026
- [26] Jitsi, “jiwer,” GitHub repository. <<https://github.com/jitsi/jiwer>> Accessed May 30, 2026